High-Order Masking for MQOM v2.1 Signing: A Baseline Design and a Rijndael LUT-Based Online Acceleration (Long Paper)

Yi-Lin Hung[1], Jiun-Peng Chen[2], Ho-Lin Chen[1] and Bo-Yin Yang[2]

[1] National Taiwan University, Taipei, Taiwan, e88888812@gmail.com, holinc@gmail.com
[2] Academia Sinica, Taipei, Taiwan, jiunpeng@ntu.edu.tw, byyang@iis.sinica.edu.tw

Abstract. This paper presents the first high-order fully-shared masking construction for MQOM v2.1, a candidate in NIST's additional digital signature standardization process. We provide a baseline high-order masked signing design for MQOM v2.1, prove its security in the standard probing leakage model, and validate the implementation through a comprehensive TVLA campaign. To mitigate the online-time bottleneck in masked signing, we further introduce an optional Rijndael LUT-based acceleration mode that decouples offline precomputation from online signing. Although this accelerated mode incurs higher offline time and memory costs, it can run during idle periods and significantly reduce online signing latency. We implement and benchmark all 36 MQOM v2.1 signing variants over GF(2), GF(16), and GF(256), and report comprehensive performance and leakage-evaluation results for both the baseline and accelerated designs.

Keywords: fully-shared masking · high-order masking · MQOM v2.1 · side-channel security · signing · TVLA

1 Introduction

If cryptographically relevant quantum computers become practical, they would threaten widely deployed public-key systems such as RSA and elliptic-curve cryptography through Shor's algorithm [RSA78, Mil86, Sho97]. In response, NIST's post-quantum cryptography effort has standardized ML-KEM and selected HQC as a backup KEM to diversify assumptions on the encryption side [Nat24a, Nat25a]. On the signature side, NIST has standardized ML-DSA and SLH-DSA, while Falcon continues a separate FN-DSA standardization track [Nat22, Nat24b, Nat24c, Nat24d]. For digital signatures, however, diversification remains an active problem, and NIST's additional digital signature process is now evaluating a second-round set of alternative candidates [Nat25b, ABC+24].

Within that process, the MPC-in-the-Head branch currently includes Mirath, MQOM, PERK, RYDE, and SDitH [AAB+25, BBFR25, ABB+25a, ABB+25b, MBB+25]. This branch is of particular interest from an implementation-security perspective. An important predecessor is Picnic, which established symmetric-cryptography-based MPC-in-the-Head signatures as practical post-quantum candidates [CDG+17]. At the same time, Picnic also showed that such designs are not automatically leakage-resilient: differential power analysis can recover the secret key from the reference implementation, and later work demonstrated that even ostensibly protected Picnic3 implementations require substantially more careful masking to avoid new attacks [GSE20, ABE+21].

MQOM v2.1 is therefore a well-motivated target for side-channel-resilient implementation work: it belongs to NIST's additional-round MPC/TCitH family, yet its masking methodology remains significantly less mature than its algorithmic design [BBFR25,

Nat25b]. Even in the absence of a dedicated public break, its repeated secret-dependent nonlinear kernels and cross-layer state reuse strongly motivate masking as a practical requirement for secure deployment.

From MQOM v1 to MQOM v2.1. A key point is that MQOM v2.1 is not merely a minor parameter update of MQOM v1: its proof framework moves to a TCitH-style threshold-sharing setting, together with revised polynomial-check and batching interfaces. In practice, this changes the structure of nonlinear operations, the locations at which masking boundaries arise, and the way randomness and commitment layers are composed. Consequently, masking arguments and engineering choices tailored to v1 do not transfer directly to v2.1.

Positioning w.r.t. prior masking work. We view *Masking-Friendly Post-Quantum Signatures in the Threshold-Computation-in-the-Head Framework* [FRWJ25] as an important reference point for TCitH-oriented masking techniques and their associated trade-offs. Our contribution is complementary: we study the concrete MQOM v2.1 signing pipeline, provide a compositional construction-to-module proof for a baseline masked design, and develop an implementation-and-TVLA validation workflow across MQOM v2.1 variants.

Our contributions. We summarize our main contributions as follows:

- We present the first high-order fully-shared masking design for MQOM v2.1. This gives a concrete end-to-end reference point for masked MQOM v2.1 signing.
- We provide a complete formal security proof for the baseline masked design. This makes the security claim explicit and checkable.
- We identify a practical masking bottleneck and introduce an optional Rijndael LUT-based acceleration mode with offline precomputation and faster online signing. This exposes a clear latency–memory trade-off for practice.
- We perform a full TVLA evaluation focused on critical small gadgets and selected composite gadgets. This adds measurement-based evidence to the formal analysis.
- We implement and benchmark all 36 MQOM v2.1 signing variants across GF(2), GF(16), and GF(256), and report comprehensive timing and trade-off results for both baseline and accelerated modes. This supports concrete parameters and deployment choices.

Paper organization. Section 2 introduces background and the threat model; Section 3 presents the baseline masked construction; Section 4 gives the optional LUT mode; Section 5 covers evaluation and deployment-relevant implications; and Section 6 concludes. Appendix A collects the formal proofs, while Appendices B and C collect detailed tables and remaining TVLA panels.

2 Preliminaries and Threat Model

2.1 MQOM v2.1 Background

We follow the MQOM v2.1 revision considered in the NIST additional-round setting and use the publicly available MQOM v2.1 specification (Version 2.1, September 22, 2025) as the baseline reference for notation and protocol structure [BBFR25]. MQOM is a Fiat–Shamir signature derived from a TCitH/MPC-in-the-Head style proof for the MQ

relation over a base field $\mathbb{F}$. The public system is represented by m quadratic equations in n variables:

$$f_i(x) = x^\top A_i x + b_i^\top x - y_i, \quad i = 1, \ldots, m,$$

where $A_i \in \mathbb{F}^{n\times n}$, $b_i \in \mathbb{F}^n$, and $y_i \in \mathbb{F}$. The witness is $x \in \mathbb{F}^n$ such that $f_i(x) = 0$ for all i.

At proof level, MQOM v2.1 uses a polynomial-check structure: the prover derives line polynomials (in the challenge variable) and commits to them through a batch line commitment (BLC) built from GGM seed trees. Two batching layers are used in the specification: packing and random-combination batching, giving the parameters N (evaluation domain size), η (internal packing dimension), and τ (external repetition count). Compared with the earlier field-embedding presentation, packing shifts arithmetic overhead from $\mathbb{F}$ to $\mathbb{K}$ while leaving the security analysis unchanged. This architectural shift is exactly why v1-oriented masking decompositions do not directly cover the v2.1 construction targeted in this paper.

In key generation, a master seed derives both the witness and a public equation seed `mseed_eq`; the latter expands deterministically to $\{A_i\}_{i=1}^m, \{b_i\}_{i=1}^m$. The public key is $(\texttt{mseed\_eq}, y)$, and the secret key stores $(\texttt{pk}, x)$. Signing samples fresh `mseed` and `salt`, computes $\texttt{com}_1 = \mathsf{BLC.Commit}(\cdot)$, derives (α_0, α_1) for the polynomial consistency relation, and binds them into $\texttt{com}_2$. The Fiat–Shamir challenge is then parsed as $i^\star \in [0, N-1]^\tau$ plus a grinding nonce `nonce`, and the signer outputs the corresponding BLC opening. Verification recomputes the same challenge from $(\texttt{pk}, \texttt{com}_1, \texttt{com}_2, \texttt{msg})$, checks grinding validity, checks BLC opening consistency, recomputes the missing coefficient share, and accepts iff all consistency/hash checks pass.

In this paper, the public protocol structure follows MQOM v2.1, while our contribution is to replace secret-dependent computations with high-order fully-shared masked counterparts, accompanied by explicit gadget-level and module-level security arguments.

We use the standard MQOM parameters and notation: λ (security level), N (evaluation-domain size), τ (external repetitions), η (internal packing dimension), and w (grinding parameter). These symbols are reused in the masked construction and proof sections below.

Core definitions used in this paper. For clarity, we fix the following protocol-level objects and interfaces.

- **Fields and domains.** $\mathbb{F}$ is the MQ base field; $\mathbb{K}$ denotes the extension field used by the polynomial-check layer; $\Omega = \{\omega_0, \ldots, \omega_{N-1}\} \subseteq \mathbb{K}$ is the evaluation domain, indexed in Gray-code order rather than by lexicographic field ordering.

- **MQ relation.** $F = (f_1, \ldots, f_m)$ with $f_i(x) = x^\top A_i x + b_i^\top x - y_i$. The witness is $x \in \mathbb{F}^n$ and the public equation descriptor is $(\texttt{mseed\_eq}, y)$.

- **Commit/challenge objects.** $\texttt{com}_1$ is the BLC commitment digest, and $\texttt{com}_2$ binds polynomial-check coefficients; $i^\star \in [0, N-1]^\tau$ and `nonce` are Fiat–Shamir challenge outputs with grinding.

- **Opening/evaluation objects.** `opening` is the BLC opening tuple; verification reconstructs $(x_{\text{eval}}, u_{\text{eval}})$ and checks consistency with (α_0, α_1).

- **Randomized seeds.** `mseed` and `salt` are per-signature randomness values; `salt` is included in signatures and provides domain separation for seed processing.

The complete unmasked protocol flow used as our masking reference is summarized in Algorithm 1.

Algorithm 1 Protocol-level view of MQOM v2.1 (unmasked reference flow).

Require: Public parameters $(\mathbb{F}, n, m, N, \tau, \eta, w)$ and hash/XOF domains

KeyGen

Sample $\texttt{seed\_key} \xleftarrow{\$} \{0,1\}^{2\lambda}$
Parse $(x, \texttt{mseed\_eq}) \leftarrow \mathsf{XOF}_0(\texttt{seed\_key})$
Expand $\{A_i\}, \{b_i\} \leftarrow \mathsf{ExpandEquations}(\texttt{mseed\_eq})$
Compute $y_i \leftarrow x^\top A_i x + b_i^\top x$ for $i = 1, \ldots, m$
Set $\texttt{pk} \leftarrow (\texttt{mseed\_eq}, y)$
Set $\texttt{sk} \leftarrow (\texttt{pk}, x)$

Sign $(\texttt{sk}, \texttt{msg})$

Parse $(\texttt{pk}, x) \leftarrow \texttt{sk}$
Parse $(\texttt{mseed\_eq}, y) \leftarrow \texttt{pk}$
Sample fresh $\texttt{mseed}, \texttt{salt} \xleftarrow{\$} \{0,1\}^{\lambda}$
$(\texttt{com}_1, \texttt{key}, x_0, u_0, u_1) \leftarrow \mathsf{BLC.Commit}(\texttt{mseed}, \texttt{salt}, x)$
$(\alpha_0, \alpha_1) \leftarrow \mathsf{ComputePAlpha}(\texttt{com}_1, x_0, u_0, u_1, x, \texttt{mseed\_eq}, y)$
$\texttt{com}_2 \leftarrow \mathsf{Hash}_3(\alpha_0, \alpha_1)$
$(i^\star, \texttt{nonce}) \leftarrow \mathsf{SampleChallenge}(\mathsf{Hash}_4(\texttt{pk}, \texttt{com}_1, \texttt{com}_2, \mathsf{Hash}_2(\texttt{msg})))$
$\texttt{opening} \leftarrow \mathsf{BLC.Open}(\texttt{key}, i^\star)$
Output $\texttt{sig} \leftarrow (\texttt{salt}, \texttt{com}_1, \texttt{com}_2, \alpha_1, \texttt{opening}, \texttt{nonce})$

Verify $(\texttt{pk}, \texttt{msg}, \texttt{sig})$

Parse $(\texttt{mseed\_eq}, y) \leftarrow \texttt{pk}$
Parse $(\texttt{salt}, \texttt{com}_1, \texttt{com}_2, \alpha_1, \texttt{opening}, \texttt{nonce}) \leftarrow \texttt{sig}$
Recompute $(i^\star, \texttt{val}) \leftarrow \mathsf{SampleChallenge}(\mathsf{Hash}_4(\texttt{pk}, \texttt{com}_1, \texttt{com}_2, \mathsf{Hash}_2(\texttt{msg})))$ with fixed nonce
$(\texttt{ret}, x_{\text{eval}}, u_{\text{eval}}) \leftarrow \mathsf{BLC.Eval}(\texttt{salt}, \texttt{com}_1, \texttt{opening}, i^\star)$
$\alpha_0 \leftarrow \mathsf{RecomputePAlpha}(\texttt{com}_1, \alpha_1, i^\star, x_{\text{eval}}, u_{\text{eval}}, \texttt{mseed\_eq}, y)$ when $\texttt{ret} \neq \perp$ (arbitrary otherwise)
$\texttt{check}_1 \leftarrow [\texttt{val} = 0]$
$\texttt{check}_2 \leftarrow [\texttt{ret} \neq \perp]$
$\texttt{check}_3 \leftarrow [\mathsf{Hash}_3(\alpha_0, \alpha_1) = \texttt{com}_2]$
Output `accept` iff $\texttt{check}_1 \wedge \texttt{check}_2 \wedge \texttt{check}_3$ (otherwise reject).

Protocol note. Key generation explicitly expands the public equations from `mseed_eq` because it directly computes y. In **Sign** and **Verify**, equation expansion is module-internal and determined by `mseed_eq`, so the protocol-level interfaces omit explicit $\{A_i\}, \{b_i\}$ arguments.

2.2 Masking Model and Security Notions

Color convention for shared state. Throughout this paper (especially in pseudocode), red-marked terms denote the share-split state. In most cases, this also means the value is a secret-dependent masked state. The main exception is at explicit public-output boundaries required by the algorithm, where shares are intentionally recombined and then treated as public outputs (e.g., commitment digests, openings, and final signature fields).

We use additive secret sharing with $d = t + 1$ shares. For a shared value $x = \sum_{i=0}^{d-1} x_i$, we denote the input-share vector by $\mathbf{x} = (x_0, \ldots, x_{d-1})$. For two shared inputs $\mathbf{x}, \mathbf{y}$ and fresh randomness $\mathbf{r}$, a gadget execution induces wire values (input, intermediate, and output wires) under the probing model.

For a gadget G and probe set P, let $\mathsf{View}_P(\mathbf{x}; \mathbf{r})$ denote the joint random variable observed on probes in P.

We also use a general bilinear map notation for multiplicative-type gadgets: $B : U \times V \to W$ is bilinear if $B(u_1 + u_2, v) = B(u_1, v) + B(u_2, v)$ and $B(u, v_1 + v_2) = B(u, v_1) + B(u, v_2)$.

t-NI (gadget level). Gadget G is t-NI if, for every $|P| \leq t$, there exists a simulator $\mathcal{S}_P$ and an index set $I \subseteq \{0, \ldots, d-1\}$ with $|I| \leq t$ such that

$$\mathsf{View}_P(\mathbf{x}; \mathbf{r}) \equiv \mathcal{S}_P\big((x_i)_{i \in I}\big)$$

for all valid shared inputs.

t-SNI (gadget level). Let a probe set contain o output probes and q non-output

probes with $o + q \leq t$. Gadget G is t-SNI if the corresponding view can be simulated using at most $q = t - o$ input shares.

t-PINI (gadget level). Gadget G is t-PINI if, for every $|P| \leq t$, the probed view can be simulated from the input shares that belong only to probed output-share domains, together with fresh randomness and public parameters (probed isolation at share-domain level).

We use the standard implication chain t-PINI $\Rightarrow$ t-SNI $\Rightarrow$ t-NI.

2.3 Evaluation Methodology

We evaluate leakage using a TVLA-style fixed-vs-random methodology on (i) core small gadgets and (ii) selected composite layers that dominate end-to-end signing behavior. For each target, we collect two trace populations under identical code path, clocking, trigger, and I/O conditions, differing only in secret class (fixed or random). Trace acquisition order is interleaved across classes to reduce drift bias.

Before statistical testing, traces are aligned with a target-specific window and a deterministic preprocessing pipeline. For first-order leakage, we apply the standard sample-wise Welch's t-test. For sample index j, with fixed-class traces F and random-class traces R, we use

$$t_j = \frac{\bar{F}_j - \bar{R}_j}{\sqrt{\frac{s^2_{F,j}}{n_F} + \frac{s^2_{R,j}}{n_R}}},$$

where $\bar{F}_j, \bar{R}_j$ are sample means, $s^2_{F,j}, s^2_{R,j}$ are sample variances, and n_F, n_R are trace counts in each class.

For each target, we report the number of traces per class, maximum absolute t-score over the analysis window, and final pass/fail decision. Unless explicitly stated otherwise, failure is declared when any sample violates $|t| > 4.5$; otherwise, the target is reported as passing under first-order TVLA. As usual, this empirical evidence complements (but does not replace) the formal probing-style full security proof in Appendix A.

3 High-Order Fully-Shared Masking for MQOM v2.1

3.1 Primitive Assumptions

We follow the masking model and security notions from Section 2.2. At the construction level, we assume:

- Fresh independent randomness for each nonlinear gadget invocation,
- probe-safe behavior for public linear transforms and share-domain conversions,
- explicit output-boundary discipline: any value that will be recombined or publicly opened is first passed through the required refresh/opening interface.

These assumptions define the contracts used by the gadget hierarchy below; the formal simulator proofs are given in Appendix A.

Construction overview. Our baseline masking stack is organized into three abstraction levels, presented across Sections 3.2–3.6. At the small-gadget layer (Section 3.2), we use bilinear gadgets and refresh gadgets as modular, nonlinear, and boundary-safe building blocks. At the arithmetic-composite layer (Section 3.3), these blocks are first composed into arithmetic kernels such as `ComputePz_mask` and `ComputePAlpha_mask`. The next integration layer then develops masked tree-expansion components (Section 3.4) and commitment/opening components (Section 3.5), including procedures such as `PRG_mask`,

GGMTree_Expand_mask, SeedCommit_mask_ctx, and BLC_Commit_mask. Finally, the full modules KeyGen_mask and Sign_mask (Sections 3.6.1 and 3.6.2) are obtained by composition with explicit output interfaces and public boundary handling.

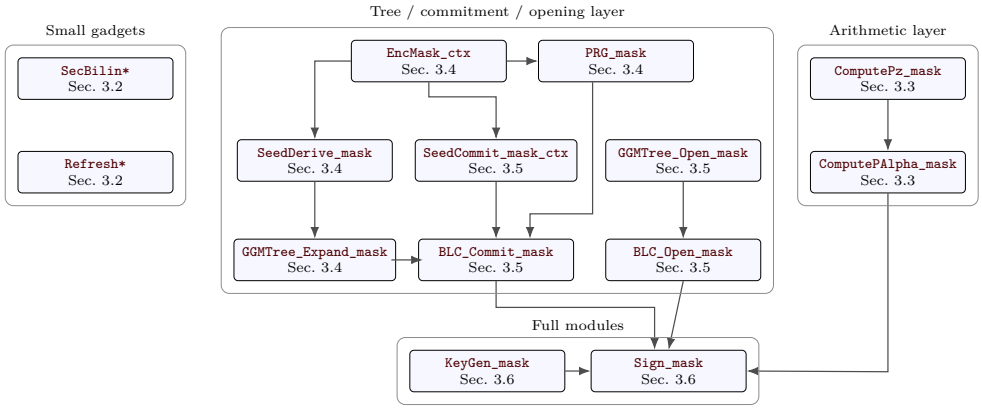


Figure 1: Module-level overview of the baseline masked stack. Directed edges indicate the main higher-level dependencies among arithmetic, tree/commitment/opening, and full-module components. The small-gadget box is shown separately as a clickable reference anchor and is intentionally left without external edges for readability.

Figure 1 gives a module-level overview of this baseline stack. It emphasizes the higher-level masked-module flow. For readability, the small-gadget box is placed separately on the left and shown only as a clickable reference anchor, without external edges.

Security-proof placement. This section focuses on construction interfaces, algorithms, and correctness-oriented composition structure. To keep the presentation concise, all formal security claims and simulator-based proofs are centralized in Appendix A; each gadget subsection below provides only a forward pointer to its corresponding formal proof subsection.

3.2 Core Gadgets

3.2.1 Small Gadget 1: General Pairwise-Masked Bilinear Gadget

Interface and functionality. Let $d = t+1$. Let $x = \sum_{i=0}^{d-1} x_i$ and $y = \sum_{i=0}^{d-1} y_i$ be additive sharings, and let $B : U \times V \to W$ be a bilinear map over fields of characteristic 2. We define the generic gadget $\mathsf{SecBilin}_B^d$ as follows.

Algorithm 2 Pseudo-algorithm for $\mathsf{SecBilin}_B^d$.

Require: Shared inputs: $(x_i)_{i=0}^{d-1}$ and $(y_i)_{i=0}^{d-1}$
Ensure: Shared outputs: $(z_i)_{i=0}^{d-1}$ with $\sum_a z_a = B(x, y)$
for $a = 0$ to $d-1$ **do**
  $z_a \leftarrow B(x_a, y_a)$
end for
for $a = 0$ to $d-1$ **do**
  for $b = a+1$ to $d-1$ **do**
    Sample $r_{a,b} \xleftarrow{\$} W$
    $z_a \leftarrow z_a + r_{a,b}$
    $z_b \leftarrow z_b + r_{a,b} + B(x_a, y_b) + B(x_b, y_a)$
  end for
end for
return $(z_i)_{i=0}^{d-1}$

Algorithm 3 Pseudo-algorithm for $\mathsf{RefreshPair}^d_{\mathbb{F}}$.

Require: Shared input: $(x_i)_{i=0}^{d-1} \in \mathbb{F}^d$
Ensure: Shared output: refreshed $(x_i)_{i=0}^{d-1}$
for $a = 0$ to $d-1$ **do**
  for $b = a+1$ to $d-1$ **do**
    Sample $r_{a,b} \xleftarrow{\$} \mathbb{F}$
    $x_a \leftarrow x_a + r_{a,b}$
    $x_b \leftarrow x_b + r_{a,b}$
  end for
end for
return $(x_i)_{i=0}^{d-1}$

Correctness. Lemma. Assume fresh independent pairwise masks $(r_{a,b})_{0 \le a < b < d}$. Then $\mathsf{SecBilin}^d_B$ is correct.

Proof. Correctness follows from bilinearity and mask cancellation in characteristic 2:

$$\sum_{a=0}^{d-1} z_a = \sum_{a=0}^{d-1} B(x_a, y_a) + \sum_{a<b} \big(B(x_a, y_b) + B(x_b, y_a)\big) = \sum_{a,b} B(x_a, y_b) = B\Big(\sum_a x_a, \sum_b y_b\Big).$$

Security proof. See Appendix A (Section A.3.1).

Cost. The gadget uses $\frac{d(d-1)}{2}$ fresh masks in W, performs d diagonal bilinear evaluations, $d(d-1)$ cross bilinear evaluations, and two share updates per unordered pair (a, b).

Instantiation corollary. Corollary. `SecBilinFieldExtExt`, `SecBilinFieldExtBase`, `SecBilinFieldBaseExt`, and `SecBilinFieldBaseBase` are instances of $\mathsf{SecBilin}^d_B$ with different bilinear maps, and therefore inherit correctness; their formal security statements are centralized in Appendix A.

3.2.2 Small Gadget 2: Pairwise Additive Share Refresh over $\mathbb{F}$

Interface and functionality. Let $d = t + 1$. For an additive sharing $x = \sum_{i=0}^{d-1} x_i$ over a finite field $\mathbb{F}$, define $\mathsf{RefreshPair}^d_{\mathbb{F}}$ by pairwise mask injection: for each unordered pair $0 \le a < b < d$, sample a fresh $r_{a,b} \xleftarrow{\$} \mathbb{F}$ and update both endpoints.

Correctness. Lemma. Let $d = t + 1$, and let $x \in \mathbb{F}$ be additively shared as $x = \sum_{i=0}^{d-1} x_i$. Then $\mathsf{RefreshPair}^d_{\mathbb{F}}$ is correct.

Proof. Correctness is immediate from pairwise cancellation in characteristic 2:

$$\sum_{i=0}^{d-1} x'_i = \sum_{i=0}^{d-1} x_i + \sum_{0 \le a < b < d} (r_{a,b} + r_{a,b}) = \sum_{i=0}^{d-1} x_i = x.$$

Security proof. See Appendix A (Section A.3.2).

Cost. The scalar gadget executes one pairwise refresh interaction per unordered pair, for a total of $\binom{d}{2} = \frac{d(d-1)}{2}$ interactions. Equivalently, this is $\frac{d(d-1)}{2}$ fresh field elements and $2\binom{d}{2} = d(d-1)$ share updates. This lemma directly instantiates to `SecRefreshFieldExtScalar` and `SecRefreshFieldBaseScalar` in the `sec_ops` module.

This same abstraction also applies to the vector refresh routines in `sec_ops`.

Algorithm 4 Pseudo-algorithm for `ComputePz_mask`.

Require: Public inputs: A, b; shared inputs: $\mathbf{x}_0, \mathbf{x}$
Ensure: Shared outputs: $(\mathbf{z}_0, \mathbf{z}_1)$

for each row i **do**
  for each share index s **do**
    $\mathbf{t}_0[s] \leftarrow A_i \cdot \mathbf{x}_0[s]$
    $\mathbf{t}_1[s] \leftarrow A_i \cdot \mathbf{x}[s]$
  end for
  $\mathbf{t}_1[0] \leftarrow \mathbf{t}_1[0] + b_i$
  $\mathbf{z}_{0,i} \leftarrow \texttt{SecBilinFieldExtExt}(\mathbf{t}_0, \mathbf{x}_0)$
  $\mathbf{t}_0 \leftarrow \texttt{SecRefreshFieldExtVec}(\mathbf{t}_0)$
  $\mathbf{x}_0 \leftarrow \texttt{SecRefreshFieldExtVec}(\mathbf{x}_0)$
  $(\mathbf{z}_{1,a}, \mathbf{z}_{1,b}) \leftarrow (\texttt{SecBilinFieldExtBase}(\mathbf{t}_0, \mathbf{x}), \texttt{SecBilinFieldBaseExt}(\mathbf{t}_1, \mathbf{x}_0))$
  $\mathbf{z}_{1,i} \leftarrow \mathbf{z}_{1,a} + \mathbf{z}_{1,b}$
end for
return $(\mathbf{z}_0, \mathbf{z}_1)$

Vector corollary. **Corollary.** Base-field and extension-field scalar/vector refresh routines inherit correctness by instantiation or coordinate-wise lift of $\mathsf{RefreshPair}^d_{\mathbb{F}}$; their formal security statements are centralized in Appendix A.

3.3 Composed Arithmetic Gadgets

3.3.1 Composite Gadget 1: `ComputePz_mask`

Interface and functionality. Let (A, b) be the public equation material, let $\mathbf{x}_0$ be a shared extension-field vector, and let $\mathbf{x}$ be a shared base-field vector. For each equation row i, define

$$\mathbf{t}_0 = A_i \cdot \mathbf{x}_0, \qquad \mathbf{t}_1 = A_i \cdot \mathbf{x} + b_i,$$

and then

$$\mathbf{z}_{0,i} = \langle \mathbf{t}_0, \mathbf{x}_0 \rangle, \qquad \mathbf{z}_{1,i} = \langle \mathbf{t}_0, \mathbf{x} \rangle + \langle \mathbf{t}_1, \mathbf{x}_0 \rangle,$$

where the inner products are realized by previously defined bilinear gadgets.

Correctness. Correctness follows from the composition of public equation material (A, b), linear maps, bilinear gadgets, and refresh gadgets. For each row i, the share-wise linear steps preserve the encoded values:

$$\sum_s \mathbf{t}_0[s] = A_i \cdot \Big(\sum_s \mathbf{x}_0[s]\Big), \qquad \sum_s \mathbf{t}_1[s] = A_i \cdot \Big(\sum_s \mathbf{x}[s]\Big) + b_i.$$

By correctness of `SecBilinFieldExtExt`,

$$\sum_s \mathbf{z}_{0,i}[s] = \langle \mathbf{t}_0, \mathbf{x}_0 \rangle.$$

Refresh preserves both shared values. By correctness of `SecBilinFieldExtBase`. By correctness of `SecBilinFieldBaseExt`,

$$\sum_s \mathbf{z}_{1,a}[s] = \langle \mathbf{t}_0, \mathbf{x} \rangle, \qquad \sum_s \mathbf{z}_{1,b}[s] = \langle \mathbf{t}_1, \mathbf{x}_0 \rangle.$$

Since the final combination is linear,

$$\sum_s \mathbf{z}_{1,i}[s] = \langle \mathbf{t}_0, \mathbf{x} \rangle + \langle \mathbf{t}_1, \mathbf{x}_0 \rangle.$$

Thus, `ComputePz_mask` computes the intended shared outputs $(\mathbf{z}_0, \mathbf{z}_1)$.

Security proof. The public row derivation from `mseed_eq` is deterministic and outside the masked boundary; see Appendix A (Section A.4.1).

Algorithm 5 Pseudo-algorithm for `ComputePAlpha_mask`.

Require: Public inputs: `mseed_eq`,com
Require: Shared inputs: $\mathbf{x}_0[0..\tau-1], \mathbf{x}, \mathbf{u}_0[0..\tau-1], \mathbf{u}_1[0..\tau-1]$
Ensure: Public outputs: (α_0, α_1)
Optionally derive public coefficients $\Gamma \leftarrow \mathsf{Gamma}(\mathsf{com})$
Set the public linear map $L_{\mathrm{com},\Gamma}$
$(A, b) \leftarrow \mathsf{ExpandEquations}(\texttt{mseed\_eq})$
for each repetition e **do**
  $(\mathbf{z}_0, \mathbf{z}_1) \leftarrow \texttt{ComputePz\_mask}(\mathbf{x}_0[e], \mathbf{x}, A, b)$
  for each share index s **do**
    $\mathbf{a}_0[s] \leftarrow L_{\mathrm{com},\Gamma}(\mathbf{z}_0[s])$
    $\mathbf{a}_1[s] \leftarrow L_{\mathrm{com},\Gamma}(\mathbf{z}_1[s])$
    $\mathbf{v}_0[s] \leftarrow \mathbf{a}_0[s] + \mathbf{u}_0[e][s]$
    $\mathbf{v}_1[s] \leftarrow \mathbf{a}_1[s] + \mathbf{u}_1[e][s]$
  end for
  $\mathbf{v}_0 \leftarrow \texttt{SecRefreshFieldExtVec}(\mathbf{v}_0)$
  $\mathbf{v}_1 \leftarrow \texttt{SecRefreshFieldExtVec}(\mathbf{v}_1)$
  $(\alpha_0[e], \alpha_1[e]) \leftarrow (\mathsf{Recombine}(\mathbf{v}_0), \mathsf{Recombine}(\mathbf{v}_1))$
end for
return (α_0, α_1)

Why refresh is essential. The shared values $\mathbf{t}_0$ and $\mathbf{x}_0$ are reused across multiple bilinear calls. Without refresh, probe information from one bilinear call can be combined with probes from later bilinear calls on the same share domains (cross-gadget leakage). The inserted `SecRefreshFieldExtVec` calls re-randomize these sharings before reuse while preserving their represented values.

Corollary. `ComputePz_mask` is a composed gadget implementing the masked computation of $(\mathbf{z}_0, \mathbf{z}_1)$ used inside `ComputePAlpha_mask`.

3.3.2 Composite Gadget 2: `ComputePAlpha_mask`

Interface and functionality. For each repetition e, `ComputePAlpha_mask` first computes shared vectors $(\mathbf{z}_0, \mathbf{z}_1)$ via `ComputePz_mask`, then applies a public linear compression map $L_{\mathrm{com},\Gamma}$, where $L_{\mathrm{com},\Gamma} = L_{\mathrm{com}}$ if statistical batching is disabled and otherwise depends on the public coefficients $\Gamma = \Gamma(\mathsf{com})$, adds masked vectors $\mathbf{u}_0[e], \mathbf{u}_1[e]$, and finally applies a last refresh before public recombination. Concretely:

The public linear map $L_{\mathrm{com},\Gamma}$ equals the fixed compression L_{com} when statistical batching is disabled; with statistical batching, it is the public $\Gamma(\mathsf{com})$-dependent linear combination. The final two refresh calls are instantiated by `SecRefreshFieldExtVec` on $\mathbf{v}_0$ and $\mathbf{v}_1$. Here, $\mathsf{Recombine}$ denotes coordinate-wise field-sum across the d shares; in the binary extension-field implementation, this is realized as share-wise XOR.

Correctness. Correctness follows by composition. For each repetition e, let the recombined values from `ComputePz_mask` be $Pz_0[e], Pz_1[e]$, so that

$$\sum_s \mathbf{z}_0[s] = Pz_0[e], \qquad \sum_s \mathbf{z}_1[s] = Pz_1[e].$$

Since $L_{\mathrm{com},\Gamma}$ is public and linear,

$$\sum_s \mathbf{a}_0[s] = L_{\mathrm{com},\Gamma}(Pz_0[e]), \qquad \sum_s \mathbf{a}_1[s] = L_{\mathrm{com},\Gamma}(Pz_1[e]).$$

Adding masked offsets is linear share-wise:

$$\sum_s \mathbf{v}_0[s] = L_{\mathrm{com},\Gamma}(Pz_0[e]) + \sum_s \mathbf{u}_0[e][s],$$

Algorithm 6 Pseudo-algorithm for `EncMask_ctx`.

Require: Shared inputs: masked context `ctx` and plaintext $\mathbf{P}$
Ensure: Shared output: ciphertext $\mathbf{C}$
1: $\mathbf{C} \leftarrow \mathsf{MaskedRijndael}_{\mathsf{ctx}}(\mathbf{P})$
2: **return** $\mathbf{C}$

$$\sum_s \mathbf{v}_1[s] = L_{\mathrm{com},\Gamma}(Pz_1[e]) + \sum_s \mathbf{u}_1[e][s].$$

The final refresh preserves the values represented by $\mathbf{v}_0$ and $\mathbf{v}_1$. Since Recombine sums shares coordinate-wise, it yields the intended outputs, $\alpha_0[e], \alpha_1[e]$, for every e.

Security proof. See Appendix A (Section A.4.2).

Why the final refresh is needed. Since α_0 and α_1 are public outputs, the boundary immediately before recombination is treated as an explicit opening boundary. The final refresh enforces the required pre-opening share normalization before the values are recombined.

Corollary. `ComputePAlpha_mask` is a derived layer built from `ComputePz_mask`, public linear post-processing, and a final refresh before opening.

Cost. Compared with the previous variant, `ComputePAlpha_mask` adds two final refresh invocations per repetition (for $\mathbf{v}_0$ and $\mathbf{v}_1$), in addition to public linear compression, linear share additions, and output recombination.

3.4 Tree and Expansion Layers

3.4.1 Composite Gadget 3: `EncMask_ctx` Wrappers

Definition (`EncMask_ctx`). Let `EncMask_ctx` denote masked encryption with a precomputed masked key schedule `ctx`. For shared plaintext block $\mathbf{P}$:

$$\begin{aligned}\mathsf{EncMask\_ctx}(\mathsf{ctx}, \mathbf{P}): \quad & \mathbf{C} \leftarrow \mathsf{MaskedRijndael}_{\mathsf{ctx}}(\mathbf{P}),\\ & \text{return } \mathbf{C}.\end{aligned}$$

Implementation-level share-layout conversions (e.g., inflate/deflate between API and backend) are omitted from the pseudo-code and treated as interface details.

API-level routines are `enc_encrypt_masked_ctx`, `enc_encrypt_masked_x2_ctx`, and `enc_encrypt_masked_x4_ctx`; the `_key` variants prepend masked key-schedule computation.

For x2/x4, the wrapper is modeled as a composition of independent single-block calls.

Correctness. By backend correctness, the wrapper computation is correct. The masked backend call correctly evaluates encryption under the masked schedule. Hence `EncMask_ctx` returns the correct sharing of $\mathsf{Enc}_k(P)$. The x2/x4 variants are repeated independent invocations, so correctness is componentwise.

Security proof. See Appendix A (Section A.5.1).

Remark on odd-share inflation. At the interface layer, API shares are adapted to internal backend shares. Since the backend does not natively support every masking order in the same layout, odd `MASK_NSHARES` uses a dedicated adaptation: one fresh random share is injected and compensated in another share before backend evaluation. This remains a linear re-sharing step, so correctness and wrapper security are preserved.

Algorithm 7 Pseudo-algorithm for `SeedDerive_mask`.

Require: Public input: $\mathsf{salt\_tweaked}$
Require: Shared input: $\mathbf{seed}$
Ensure: Shared output: $\mathbf{out}$
1: $\psi \leftarrow \mathsf{LinOrtho}(\mathbf{seed})$
2: $\mathbf{c} \leftarrow \mathsf{MaskedEnc}_{\mathsf{salt\_tweaked}}(\mathbf{seed})$
3: $\mathbf{out} \leftarrow \mathbf{c} + \psi$
4: **return** $\mathbf{out}$

Algorithm 8 Pseudo-algorithm for `PRG_mask`.

Input: masked seed $\mathbf{seed}$, public salt salt,
execution index $e \in [0, \tau - 1]$, byte count $n_{\text{bytes}} \in \mathbb{N}$
Output: masked pseudorandom bytes $\mathbf{out} \in \{0,1\}^{8 \cdot n_{\text{bytes}}}$
1: $n_{\text{blocks}} \leftarrow \left\lceil \frac{8 \cdot n_{\text{bytes}}}{\lambda} \right\rceil$
2: $\psi \leftarrow \mathsf{LinOrtho}(\mathbf{seed})$
3: **for** $j = 0$ to $n_{\text{blocks}} - 1$ **do**
4: $\quad \mathsf{tweaked\_salt} \leftarrow \mathsf{TweakSalt}(\mathsf{salt}, 3, e, j)$
5: $\quad \mathbf{block}[j] \leftarrow \mathsf{MaskedEnc}_{\mathsf{tweaked\_salt}}(\mathbf{seed}) + \psi$
6: **end for**
7: $(\mathbf{byte}[i])_{i=0}^{n_{\text{blocks}} \cdot \lambda/8 - 1} \leftarrow \mathsf{ParseBytes}((\mathbf{block}[j])_{j=0}^{n_{\text{blocks}} - 1})$
8: $\mathbf{out} \leftarrow \mathsf{TakeBytes}((\mathbf{byte}[i])_{i=0}^{n_{\text{bytes}} - 1})$
9: **return** $\mathbf{out}$

3.4.2 Composite Gadget 4: `SeedDerive_mask`

Definition (`SeedDerive_mask`). The masked seed-derivation gadget takes a shared seed and a public tweaked salt and computes

$$\begin{aligned} \mathsf{SeedDerive\_mask}(\mathsf{salt\_tweaked}, \mathbf{seed}): \quad & \psi \leftarrow \mathsf{LinOrtho}(\mathbf{seed}), \\ & \mathbf{c} \leftarrow \mathsf{MaskedEnc}_{\mathsf{salt\_tweaked}}(\mathbf{seed}), \\ & \text{return } \mathbf{c} + \psi. \end{aligned}$$

Here `LinOrtho` is a public linear transform, `MaskedEnc` denotes masked block-cipher evaluation under the tweaked salt (instantiated by `enc_encrypt_masked_ctx`), and $+$ is share-wise XOR.

Correctness. `LinOrtho` is a deterministic public linear. `MaskedEnc` correctly encrypts the shared seed. Their share-wise XOR yields the intended derived seed.

Security proof. See Appendix A (Section A.5.2).

Corollary (batched variants). The x2/x4 batched routines are parallel, independent `SeedDerive_mask` evaluations and inherit the same correctness property.

3.4.3 Derived Layer: `PRG_mask`

Definition (`PRG_mask`). Let $\mathbf{seed}$ be a shared seed. For each block index j, define

$$\mathbf{block}_j = \mathsf{MaskedEnc}_{\mathsf{TweakSalt}(\mathsf{salt},3,e,j)}(\mathbf{seed}) + \mathsf{LinOrtho}(\mathbf{seed}).$$

`PRG_mask` outputs the concatenation of the requested number of such blocks, truncated if needed.

The x2/x4 calls are batching optimizations only: they evaluate several independent block instances in parallel.

Algorithm 9 Pseudo-algorithm for `GGMTree_Expand_mask`.

Require: Public inputs: (salt, e); shared inputs: $\mathbf{rseed}, \delta$
Ensure: Shared outputs: tree nodes **node** and leaves **lseed**
1: $\mathbf{node}[2] \leftarrow \mathbf{rseed}$
2: $\mathbf{node}[3] \leftarrow \mathbf{rseed} + \delta$
3: **for** each tree level j **do**
4: $\quad \mathsf{ctx} \leftarrow \mathsf{KeySchedule}(\mathsf{TweakSalt}(\mathsf{salt}, 2, e, j-1))$
5: $\quad$ **for** each active parent index k at level j **do**
6: $\quad\quad \mathbf{node}[2k] \leftarrow \mathsf{SeedDerive\_mask}_{\mathsf{ctx}}(\mathbf{node}[k])$
7: $\quad\quad \mathbf{node}[2k+1] \leftarrow \mathbf{node}[2k] + \mathbf{node}[k]$
8: $\quad$ **end for**
9: **end for**
10: Copy leaf region of **node** into **lseed**
11: **return** (**node**, **lseed**)

Correctness. Masked-encryption correctness and linearity imply correctness of `PRG_mask`. `LinOrtho` is a deterministic public linear. Each masked-encryption call returns a correct sharing under the corresponding tweaked public key. Adding ψ share-wise yields the intended block definition. Concatenation and truncation are output-formatting steps only. Hence, `PRG_mask` correctly realizes the intended masked PRG expansion.

Security proof. See Appendix A (Section A.5.3).

Remark. `PRG_mask` is not treated as a new nonlinear masking primitive. It is a derived layer built from repeated invocations of masked encryption with public-domain-separated tweaks, followed only by linear post-processing.

Composition notes. For higher-level modules (e.g., `BLC_Commit_mask`), we use `PRG_mask`, `GGMTree_Expand_mask`, and `SeedCommit_mask_ctx` as derived subcomponents; their formal security statements are centralized in Appendix A.

3.4.4 Composite Gadget 6: `GGMTree_Expand_mask`

Definition (`GGMTree_Expand_mask`). `GGMTree_Expand_mask` expands a masked root seed **rseed** and masked offset δ into a full shared GGM tree. It initializes

$$\mathbf{node}[2] \leftarrow \mathbf{rseed}, \qquad \mathbf{node}[3] \leftarrow \mathbf{rseed} + \delta,$$

and then derives children level by level:

$$\mathbf{node}[2k] \leftarrow \mathsf{SeedDerive\_mask}(\mathbf{node}[k]), \qquad \mathbf{node}[2k+1] \leftarrow \mathbf{node}[2k] + \mathbf{node}[k].$$

Finally, leaf nodes are copied into the output array **lseed**.

Correctness. Correctness follows by induction on tree depth. Initialization of $\mathbf{node}[2]$ and $\mathbf{node}[3]$ is correct by construction. By the correctness of `SeedDerive_mask`, each left child $\mathbf{node}[2k]$ is correctly derived from its parent. The right child $\mathbf{node}[2k+1] = \mathbf{node}[2k] + \mathbf{node}[k]$ is obtained via a share-wise linear computation and is therefore correct. Repeating level by level proves the correctness of the entire tree. Copying the leaf region into **lseed** preserves values.

Security proof. See Appendix A (Section A.5.4).

Important notes (x2/x4 batching). The x2/x4 batched derivation paths are optimization-only: they evaluate several independent derivations in parallel and do not alter the masking structure or proof argument.

Algorithm 10 Pseudo-algorithm for `SeedCommit_mask_ctx`.

Require: Public inputs: $\mathsf{ctx}_1, \mathsf{ctx}_2$
Require: Shared input: **seed**
Ensure: Public output: $\mathsf{out}_1 \parallel \mathsf{out}_2$
1: $\psi \leftarrow \mathsf{LinOrtho}(\mathbf{seed})$
2: $(\mathbf{c}_1, \mathbf{c}_2) \leftarrow \mathsf{MaskedEnc\_x2\_ctx}(\mathsf{ctx}_1, \mathsf{ctx}_2, \mathbf{seed}, \mathbf{seed})$
3: $(\mathbf{c}_1, \mathbf{c}_2) \leftarrow (\mathbf{c}_1 + \psi, \mathbf{c}_2 + \psi)$
4: $(\mathbf{c}_1, \mathbf{c}_2) \leftarrow (\mathsf{RefreshPair}^d_{\mathbb{F}}(\mathbf{c}_1), \mathsf{RefreshPair}^d_{\mathbb{F}}(\mathbf{c}_2))$
5: $(\mathsf{out}_1, \mathsf{out}_2) \leftarrow (\mathsf{Recombine}(\mathbf{c}_1), \mathsf{Recombine}(\mathbf{c}_2))$
6: **return** $\mathsf{out}_1 \parallel \mathsf{out}_2$

Corollary. `GGMTree_Expand_mask` is constructed from `SeedDerive_mask` and linear share-wise updates, and can therefore be incorporated into `BLC_Commit_mask`.

3.5 Commitment and Opening Layers

3.5.1 Composite Gadget 5: `SeedCommit_mask_ctx`

Definition (`SeedCommit_mask_ctx`). `SeedCommit_mask_ctx` takes a shared seed and two masked encryption contexts, and computes a two-block commitment: First, it derives $\psi \leftarrow \mathsf{LinOrtho}(\mathbf{seed})$ and computes $(\mathbf{c}_1, \mathbf{c}_2) \leftarrow \mathsf{MaskedEnc\_x2\_ctx}(\mathsf{ctx}_1, \mathsf{ctx}_2, \mathbf{seed}, \mathbf{seed})$. It then refreshes both ciphertext blocks at the output boundary:

$$\mathbf{c}_j \leftarrow \mathsf{RefreshPair}^d_{\mathbb{F}}(\mathbf{c}_j + \psi), \qquad j \in \{1, 2\}.$$

Finally, it returns $\mathsf{Recombine}(\mathbf{c}_1) \parallel \mathsf{Recombine}(\mathbf{c}_2)$. This is exactly the masked analog of the unmasked seed-commitment computation, with a final output-boundary refresh before recombination.

Correctness. Correctness follows from the correctness of masked encryption and linearity. `LinOrtho` is a deterministic public linear transform of the seed. `MaskedEnc_x2_ctx` correctly computes two masked ciphertext blocks for the same seed under ctx_1 and ctx_2. Adding ψ to each ciphertext block is share-wise linear. The final refresh preserves represented values, so recombining each refreshed block yields the intended public commitment blocks. Hence, `SeedCommit_mask_ctx` computes the correct commitment.

Security proof. See Appendix A (Section A.6.1).

Corollary (batched helper). `SeedCommit_mask_x2_ctx` is two independent invocations of `SeedCommit_mask_ctx` and inherits the same correctness property.

3.5.2 Opening Routine: `GGMTree_Open_mask`

Definition (`GGMTree_Open_mask`). `GGMTree_Open_mask` takes a shared GGM tree and a public challenge index $i^\star$, then outputs the authentication path by recombining sibling nodes along the corresponding leaf-to-root path.

Correctness. Correctness is immediate: on each level, the routine outputs exactly the sibling node required by the protocol-defined authentication path for challenge $i^\star$.

Formal-security positioning. Formal treatment is provided in Section A.6.2. At this layer, `GGMTree_Open_mask` is modeled as a protocol-opening interface (authorized public output), rather than as a gadget-level t-NI/t-SNI/t-PINI security target. The corresponding masking guarantee is attached to the upstream producer `GGMTree_Expand_mask`.

Algorithm 11 Pseudo-algorithm for `GGMTree_Open_mask`.

Require: Public input: $i^\star$
Require: Shared input: **node**
Ensure: Public output: path
1: $i \leftarrow N + i^\star$
2: **for** $j = 0$ to $\log(N) - 1$ **do**
3: path$[j] \leftarrow$ Recombine(**node**$[i \oplus 1]$)
4: $i \leftarrow \lfloor i/2 \rfloor$
5: **end for**
6: **return** path

3.5.3 Composite Gadget 7: `BLC_Commit_mask`

Definition (`BLC_Commit_mask`). `BLC_Commit_mask` takes shared **mseed**, public salt, and shared witness $\mathbf{x}$, and computes a public digest com_1, an auxiliary opening key key, and masked outputs $(\mathbf{x}_0, \mathbf{u}_0, \mathbf{u}_1)$. At a high level, it derives root seeds by calling `PRG_mask` with root-domain parameters, builds masked trees via `GGMTree_Expand_mask`, computes public leaf commitments via `SeedCommit_mask_ctx`, expands leaf seeds again with `PRG_mask`, then parses leaves, performs Gray-code folding, and accumulates weighted coefficients with public weights $w_j = 2^j$ before final public hashing of the retained partial-delta bytes. The pseudocode below keeps mathematical arrays explicit and introduces key only at the final serialization step.

Correctness. Correctness follows by composition. `PRG_mask` correctly derives masked root/leaf expansions, `GGMTree_Expand_mask` correctly derives shared tree and leaf seeds, `SeedCommit_mask_ctx` correctly computes public leaf commitments, and parse/accumulate steps are linear and correctness-preserving. The final digest computation is a public hashing over designated public commitment material. Hence `BLC_Commit_mask` correctly returns the commitment digest and the masked values for subsequent layers.

Security proof. See Appendix A (Section A.6.3).

Important notes (output boundary). The digest com_1 is an authorized public protocol output. Hence, formal security analysis is attached to the masked coefficient-generation stage and internal commitment processing before the public hashing boundary.

3.5.4 Opening Routine: `BLC_Open_mask`

Definition (`BLC_Open_mask`). `BLC_Open_mask` takes auxiliary opening key produced during commitment and public challenge indices $\mathsf{i}^\star$. It outputs the verifier-required public opening data. Write key $= (\mathbf{node}, \mathsf{lsCom}, \mathsf{pDeltaX})$. For each repetition index e, it:

- computes path_e by applying `GGMTree_Open_mask` to shared node state and $\mathsf{i}^\star[e]$,
- sets outCom_e as the challenged public leaf commitment from $\mathsf{lsCom}[e][\mathsf{i}^\star[e]]$,
- sets pDelta_e as the corresponding public partial-delta value from $\mathsf{pDeltaX}[e]$.

It returns the concatenation of path_e, outCom_e, and pDelta_e for all e.

This matches the protocol-opening structure exactly: authentication-path extraction from masked node state, plus direct selection of corresponding public commitment and partial-delta entries.

Correctness. Correctness is immediate from the correctness of the stored auxiliary state:

- `node[e]` contains the masked GGM tree produced by `BLC_Commit_mask`,

- `GGMTree_Open_mask` correctly returns the challenged authentication path for `node[e]` and `i_star[e]`,
- `ls_com[e][i_star[e]]` is exactly the public commitment of that leaf,
- `partial_delta_x[e]` is copied unchanged.

Therefore, `BLC_Open_mask` outputs exactly the protocol-specified opening for challenge indices `i_star`.

Algorithm 12 Pseudo-algorithm for `BLC_Commit_mask`.

Input: masked master seed $\mathbf{mseed}$, public salt salt, masked witness $\mathbf{x}$
Output: public commitment com_1, auxiliary opening key key, masked coefficients $\mathbf{x}_0, \mathbf{u}_0, \mathbf{u}_1$

$(\mathbf{rseed}[e])_{e=0}^{\tau-1} \leftarrow \mathsf{ParseRootSeeds}(\texttt{PRG\_mask}(\mathsf{tree\_prg\_salt}, 0, \tau \cdot \texttt{SEED\_SIZE}, \mathbf{mseed}))$
$\delta \leftarrow \mathsf{FirstBytes}_\lambda(\mathsf{SerializeShares}(\mathbf{x}))$
for $e = 0$ to $\tau - 1$ **do**
  $(\mathbf{node}[e], \mathbf{lseed}[e]) \leftarrow \texttt{GGMTree\_Expand\_mask}(\mathsf{salt}, \mathbf{rseed}[e], \delta, e)$
  $(\mathsf{tweaked\_salt}_{0,e}, \mathsf{tweaked\_salt}_{1,e}) \leftarrow (\mathsf{TweakSalt}(\mathsf{salt}, 0, e, 0), \mathsf{TweakSalt}(\mathsf{salt}, 1, e, 0))$
  $(\mathsf{ctx}_{1,e}, \mathsf{ctx}_{2,e}) \leftarrow (\mathsf{KeySchedule}(\mathsf{tweaked\_salt}_{0,e}), \mathsf{KeySchedule}(\mathsf{tweaked\_salt}_{1,e}))$
  $\mathsf{hashCtx}_e \leftarrow \mathsf{HashInit}(6)$
  $\mathbf{accX}[e] \leftarrow 0,\ \mathbf{x}_0[e] \leftarrow 0,\ \mathbf{u}_0[e] \leftarrow 0,\ \mathbf{u}_1[e] \leftarrow 0$
  for $i = 0$ to $N - 1$ step 2 **do**
    $(\mathsf{lsCom}[e][i], \mathsf{lsCom}[e][i+1]) \leftarrow \texttt{SeedCommit\_mask\_x2\_ctx}(\mathsf{ctx}_{1,e}, \mathsf{ctx}_{2,e}, \mathbf{lseed}[e][i], \mathbf{lseed}[e][i+1])$
    for $b = 0$ to 1 **do**
      $\mathbf{exp}[e][i+b] \leftarrow \texttt{PRG\_mask}(\mathsf{salt}, e, i+b, \mathbf{lseed}[e][i+b])$
      $(\bar{\mathbf{x}}_{e,i+b}, \bar{\mathbf{u}}_{e,i+b}) \leftarrow \mathsf{Parse}(\mathbf{lseed}[e][i+b] \,\|\, \mathbf{exp}[e][i+b])$
      $\mathsf{hashCtx}_e \leftarrow \mathsf{HashUpdate}(\mathsf{hashCtx}_e, \mathsf{lsCom}[e][i+b])$
      $(\mathbf{u}_1[e], \mathbf{u}_0[e]) \leftarrow (\mathbf{u}_1[e] + \bar{\mathbf{u}}_{e,i+b}, \mathbf{u}_0[e] + \omega_{i+b} \cdot \bar{\mathbf{u}}_{e,i+b})$
      $(\mathbf{accX}[e], \mathbf{x}_0[e]) \leftarrow (\mathbf{accX}[e] + \bar{\mathbf{x}}_{e,i+b}, \mathbf{x}_0[e] + \omega_{i+b} \cdot \bar{\mathbf{x}}_{e,i+b})$
    end for
  end for
  $\mathsf{hashLsCom}[e] \leftarrow \mathsf{HashFinalize}(\mathsf{hashCtx}_e)$
  $\Delta x[e] \leftarrow \mathsf{SecRefreshFieldBaseVec}(\mathbf{x} + \mathbf{accX}[e])$
  $\mathsf{serDeltaX}[e] \leftarrow \bigoplus_{s=0}^{d-1} \mathsf{Serialize}(\Delta x[e][s])$
  $\mathsf{pDeltaX}[e] \leftarrow \mathsf{NextBits}(\mathsf{serDeltaX}[e])$
end for
$\mathsf{com}_1 \leftarrow \mathsf{Hash}_7(\mathsf{hashLsCom}, \mathsf{pDeltaX})$
$\mathsf{key} \leftarrow \mathsf{SerializeOpenState}((\mathbf{node}[e])_{e=0}^{\tau-1}, \mathsf{lsCom}, \mathsf{pDeltaX})$
return $(\mathsf{com}_1, \mathsf{key}, \mathbf{x}_0, \mathbf{u}_0, \mathbf{u}_1)$

Algorithm 13 Pseudo-algorithm for `BLC_Open_mask`.

Require: Shared auxiliary input: key
Require: Public input: $\mathsf{i}^\star$
Ensure: Public output: opening data

$(\mathbf{node}, \mathsf{lsCom}, \mathsf{pDeltaX}) \leftarrow \mathsf{ParseOpenKey}(\mathsf{key})$
for each repetition e **do**
  $\mathsf{path}_e \leftarrow \texttt{GGMTree\_Open\_mask}(\mathbf{node}[e], \mathsf{i}^\star[e])$
  $\mathsf{outCom}_e \leftarrow \mathsf{lsCom}[e][\mathsf{i}^\star[e]]$
  $\mathsf{pDelta}_e \leftarrow \mathsf{pDeltaX}[e]$
end for
return $\mathsf{Concat}(\mathsf{path}_e, \mathsf{outCom}_e, \mathsf{pDelta}_e \text{ for all } e)$

Algorithm 14 Construction-level pseudocode for `KeyGen_mask`.

Input: masked seed `seed_key`
Output: public key `pk`, masked secret key `sk`

$(\mathbf{x}, \mathbf{pkseed}) \leftarrow$ XOF_mask_Parse(`seed_key`)
`sk->x_part` $\leftarrow$ SerializeMasked($\mathbf{x}$)
`pk`$[0..2 \cdot$ `SEED_SIZE` $- 1] \leftarrow$ Recombine($\mathbf{pkseed}$)
$(A, b) \leftarrow$ ExpandEquations(`pk`$[0..2 \cdot$ `SEED_SIZE` $- 1]$)
for each equation index i **do**
  for each share s **do**
    $\mathbf{t}_i[s] \leftarrow A_i \cdot \mathbf{x}[s]$
  end for
  $\mathbf{y}[i] \leftarrow$ `SecBilinFieldBaseExt`$(\mathbf{x}, \mathbf{t}_i)$
  $\mathbf{x} \leftarrow$ `SecRefreshFieldBaseVec`$(\mathbf{x})$
  for each share s **do**
    $\mathbf{y}[i][s] \leftarrow \mathbf{y}[i][s] + \langle b_i, \mathbf{x}[s] \rangle$
  end for
  $\mathbf{y}[i] \leftarrow$ `SecRefreshFieldExtScalar`$(\mathbf{y}[i])$
  $y[i] \leftarrow$ Recombine($\mathbf{y}[i]$)
end for
`pk` $\leftarrow$ SerializePublicKey(`pk`$[0..2 \cdot$ `SEED_SIZE` $- 1], y)$
`sk->pk_part` $\leftarrow$ `pk`
return (`pk`, `sk`)

Formal-security positioning. Formal treatment is given in Section A.6.4. At this layer, `BLC_Open_mask` is treated as an authorized output-opening interface for protocol-visible data, not as an independent gadget-level t-NI/t-SNI/t-PINI claim. The masking claim remains attached to the input-producing commitment gadget `BLC_Commit_mask` (Section A.6.3).

3.6 KeyGen_mask and Sign_mask (Construction View)

3.6.1 `KeyGen_mask` (construction-level module view)

Module binding. This module corresponds to `keygen_mask`.

Interface and role. `KeyGen_mask` takes a masked `seed_key` and returns the public `pk` and the masked `sk`. Operationally, it performs masked seed expansion, followed by masked quadratic evaluation, then opens only the designated public-key boundary. Here each $\mathbf{t}_i[s]$ is an extension-field vector obtained by the public base-to-extension multiplication $A_i \cdot \mathbf{x}[s]$.

Correctness/proof link. `KeyGen_mask` correctness is by composition; the formal module-level proof is deferred to Appendix A (Section A.7.1).

3.6.2 `Sign_mask` (construction-level module view)

Module binding. This module corresponds to `sign_mask`.

Interface and role. `Sign_mask` takes masked `sk`, a fresh masked per-signature seed `mseed`, and public `msg`/`salt`, and outputs public signature `sig`. It reads the embedded public-key copy `sk->pk_part` to recover `pk` and `mseed_eq`, then composes commitment, arithmetic, explicit Hash_2/Hash_3/Hash_4 challenge grinding, and authorized opening.

Correctness/proof link. `Sign_mask` correctness is by composition; the formal module-level proof is deferred to Appendix A (Section A.7.2).

Algorithm 15 Construction-level pseudocode for `Sign_mask`.

Input: masked secret key $\mathtt{sk}$, public message `msg`, public `salt`, masked per-signature seed $\mathtt{mseed}$
Output: public signature `sig`

$\mathbf{x} \leftarrow \mathsf{ParseMaskedSecretKey}(\mathtt{sk})$
$\mathtt{pk} \leftarrow \mathtt{sk->pk\_part}, \quad \mathtt{mseed\_eq} \leftarrow \mathtt{pk}[0..2 \cdot \mathtt{SEED\_SIZE} - 1]$
$(\mathtt{com1}, \mathtt{key}, \mathbf{x0}, \mathbf{u0}, \mathbf{u1}) \leftarrow \mathtt{BLC\_Commit\_mask}(\mathtt{mseed}, \mathtt{salt}, \mathbf{x})$
$(\mathtt{alpha0}, \mathtt{alpha1}) \leftarrow \mathtt{ComputePAlpha\_mask}(\mathtt{mseed\_eq}, \mathtt{com1}, \mathbf{x0}, \mathbf{x}, \mathbf{u0}, \mathbf{u1})$
$\mathtt{com2} \leftarrow \mathsf{Hash}_3(\mathtt{alpha0}, \mathtt{alpha1})$
$\mathtt{msg\_hash} \leftarrow \mathsf{Hash}_2(\mathtt{msg})$
$\mathtt{hash} \leftarrow \mathsf{Hash}_4(\mathtt{pk}, \mathtt{com1}, \mathtt{com2}, \mathtt{msg\_hash})$
$(\mathtt{i\_star}, \mathtt{nonce}) \leftarrow \mathsf{SampleChallenge}(\mathtt{hash})$
$\mathtt{opening} \leftarrow \mathtt{BLC\_Open\_mask}(\mathtt{key}, \mathtt{i\_star})$
$\mathtt{sig} \leftarrow (\mathtt{salt}, \mathtt{com1}, \mathtt{com2}, \mathtt{alpha1}, \mathtt{opening}, \mathtt{nonce})$
return `sig`

4 Optional Rijndael LUT-Based Online Acceleration for MQOM v2.1 Signing

4.1 Motivation: Masking Bottleneck in Signing

The main bottleneck of baseline masked signing is the repeated masked Rijndael-based seed derivation and PRG expansion within the BLC commitment path. In our empirical study, the highest cost does not occur uniformly across all signing modules; it is concentrated in Rijndael-heavy masked subroutines that are repeatedly invoked by `SeedDerive_mask`, `PRG_mask`, and dependent commitment-flow components.

Accordingly, the optional LUT mode is not introduced merely to replace arithmetic by lookups. Its core purpose is to move one-time masked Rijndael preparation from the online signing path into an offline precomputation phase. In practice, the decisive optimization is not the mere existence of LUTs, but whether the one-time PRG/table-generation state is fully removed from online signing.

4.2 Acceleration Architecture

The accelerated mode is implemented as an optional backend specialization of the masked signing path. The outer signing interface and transcript semantics are unchanged; only selected AES-heavy subroutines inside the commitment/derivation flow are redirected to LUT-assisted masked Rijndael components.

At the protocol level, `Sign_mask` and `Sign_mask_lut` have identical transcript semantics: the same BLC commitment, PIOP/α computation, challenge derivation, and opening steps are executed. The acceleration is purely backend-local to the commitment hot path, replacing selected PRG/tree/leaf-commit Rijndael-heavy subroutines with LUT-backed versions. A public one-time context `preCtx`, built offline by `LUT_Prebuild`, stores domain-indexed LUT/PRG/round-key material that is then consumed online by `BLC_Commit_mask_lut` and `Sign_mask_lut` without changing any protocol-visible output. Algorithms 16–22 summarize this accelerated path.

Figure 2 gives a module-level view of this split execution model. The offline phase materializes a public precomputation context once, while the online phase reuses that context only along the Rijndael-heavy commitment path. Arithmetic-side modules such as `ComputePz_mask` and `ComputePAlpha_mask`, as well as the opening path `BLC_Open_mask`, remain unchanged from the baseline construction.

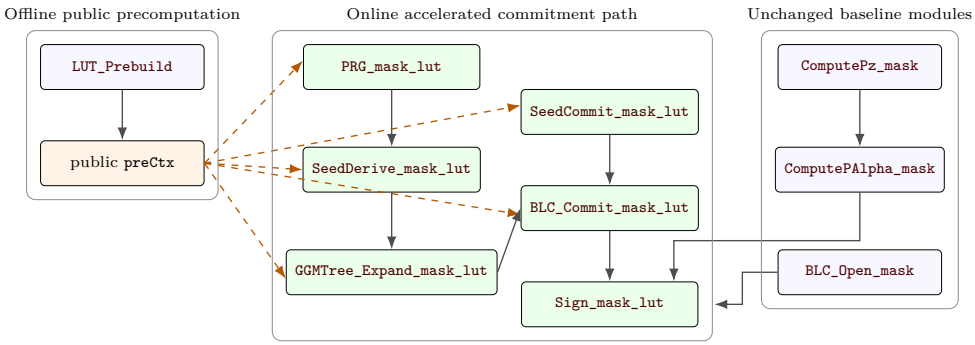


Figure 2: Module-level overview of the LUT-accelerated signing path. The offline phase builds a public context `preCtx`; dashed edges indicate public-context consumption during online signing. Only the Rijndael-heavy commitment path is specialized, while arithmetic-side modules and the opening routine remain unchanged from the baseline construction.

Algorithm 16 Pseudo-algorithm for `LUT_Prebuild`.

Public input: backend parameters, share order d, public salt-domain parameters
Shared input: none
Offline public precomputed state: `preCtx`
Public output: `preCtx`

Construct public tweak-domain descriptor set:
  D_{root}, $D_{\mathsf{derive}}(e,j) = \mathsf{TweakSalt}(\mathsf{salt}, 2, e, j)$, $D_{\mathsf{commit}}(e,0) = \mathsf{TweakSalt}(\mathsf{salt}, 0, e, 0)$,
  $D_{\mathsf{commit}}(e,1) = \mathsf{TweakSalt}(\mathsf{salt}, 1, e, 0)$, and $D_{\mathsf{leaf}}(e) = \mathsf{TweakSalt}(\mathsf{salt}, 3, e, \cdot)$
for each public domain descriptor `f` in $\{D_{\mathsf{root}}, D_{\mathsf{derive}}, D_{\mathsf{commit}}, D_{\mathsf{leaf}}\}$ **do**
  `preCtx.tweak[f]` ← instantiate public tweak/tweak-salt parameters for `f`
  `preCtx.table[f]` ← build one-time LUT/table state for `f`
  `preCtx.prg[f]` ← build one-time PRG-related state for `f` when applicable
  `preCtx.rk[f]` ← build one-time round-key-related state for `f` when applicable
end for
`preCtx.meta` ← store domain-to-state mapping, indexing, layout, and dimension metadata
return `preCtx`

Algorithm 17 Pseudo-algorithm for `PRG_mask_lut`.

Public input: public salt-domain parameters, offline public context `preCtx`
Shared input: masked seed `seed`
Shared output: masked expansion block `exp`

Select public domain descriptor `f` (root/derive/leaf) from `preCtx.meta`
Load `preCtx.tweak[f]`, `preCtx.table[f]`, `preCtx.prg[f]`, and `preCtx.rk[f]`
Evaluate LUT-backed masked PRG expansion on `seed`
return `exp`

Algorithm 18 Pseudo-algorithm for `SeedDerive_mask_lut`.

Public input: public salt-domain parameters, offline public context `preCtx`
Shared input: masked seed `seed`
Shared output: masked derived seed `out`

Select public domain descriptor `f` for GGM derivation from `preCtx.meta`
Load `preCtx.tweak[f]`, `preCtx.table[f]`, and `preCtx.rk[f]`
`out` ← evaluate LUT-backed masked seed-derivation computation on `seed`
return `out`

The concrete compact layout of `preCtx` is deferred to Section 4.3, where the state-ownership split is summarized.

The per-seed derivation core in the LUT path is `SeedDerive_mask_lut`. Inside tree expansion, this core is consumed through x2/x4 context-batched wrappers.

Algorithm 19 Pseudo-algorithm for `GGMTree_Expand_mask_lut`.

Public input: offline public context `preCtx`
Shared input: masked root seed `rseed`, masked offset δ
Shared output: masked tree nodes `node`, masked leaf seeds `lseed`

`node`[2] ← `rseed`
`node`[3] ← `rseed` ⊕ δ
for $j = 1$ to $\log_2(N) - 1$ **do**
    `ctx_j` ← `preCtx.levelCtx`[j-1]
    Derive level-j children in x2/x4 batches using `ctx_j`
    . . . batched wrappers of `SeedDerive_mask_lut` on `node` parents; same child-XOR completion as in `GGMTree_Expand_mask` . . .
end for
Copy leaf region of `node` into `lseed`
return (`node`, `lseed`)

Algorithm 20 Pseudo-algorithm for `SeedCommit_mask_lut`.

Public input: public salt-domain parameters, offline public context `preCtx`
Shared input: masked seed `seed`
Public output: leaf commitment `ls_com`

Select public domain descriptor `f` for commit branch 0 or 1 from `preCtx.meta`
Load `preCtx.tweak[f]`, `preCtx.table[f]`, and `preCtx.rk[f]`
Evaluate LUT-backed masked seed-commit computation on `seed`
return `ls_com`

Algorithm 21 Pseudo-algorithm for `BLC_Commit_mask_lut`.

Public input: public salt `salt`, offline public context `preCtx`
Shared input: masked master seed `mseed`, masked witness `x`
Output: commitment `com1`, opening key `key`, masked coefficients `x0`, `u0`, `u1`

$(\texttt{rseed}[e])_{e=0}^{\tau-1}$ ← `ParseRootSeeds`(`PRG_mask_lut`(`preCtx.rseed_prg`, `mseed`))
δ ← `FirstBytes_lambda`(`SerializeShares`(`x`))
for $e = 0$ to $\tau - 1$ **do**
    (`node`[e], `lseed`[e]) ← `GGMTree_Expand_mask_lut`(`preCtx.ggm`[e], `rseed`[e], δ)
    `hashCtx_e` ← `HashInit`(`domain = 6`); `accX`[e], `x0`[e], `u0`[e], `u1`[e] ← 0
    for $i = 0$ to $N - 1$ step 2 **do**
        (`ctx1`, `ctx2`) ← (`preCtx.seedcommit_ctx1`[e], `preCtx.seedcommit_ctx2`[e]);
    (`ls_com`[e][i], `ls_com`[e][i+1]) ← `SeedCommit_mask_lut_x2_ctx`(`ctx1`, `ctx2`, `lseed`[e][i], `lseed`[e][i+1])
        . . . same masked parse/hash/accumulate updates as in `BLC_Commit_mask` . . .
    end for
    . . . same `hashLsCom/pDeltaX` derivation as in `BLC_Commit_mask` . . .
end for
`com1` ← `PublicHash`(`hashLsCom`, `pDeltaX`)
`key` ← `Serialize`(`node`, `ls_com`, `pDeltaX`)
return (`com1`, `key`, `x0`, `u0`, `u1`)

In this interface split, `preCtx` is a public one-time backend state, while `rseed`, δ, `node`, `key`, and `lseed` are online secret-dependent masked state.

4.3 Optional Offline/Online Signing Split

A meaningful offline/online split is achieved only when the online phase performs neither LUT generation nor PRG-state construction. Moving lookup tables alone is insufficient if the PRG state is still rebuilt online.

Offline owner. In the LUT path, `LUT_Prebuild` is the key offline owner routine. Its role is to construct one-time public acceleration material independent of the witness-level

Algorithm 22 Pseudo-algorithm for `Sign_mask_lut`.

Public input: message `msg`, public salt `salt`, offline public context `preCtx`
Shared input: masked secret key `sk`, masked per-signature seed `mseed`
Public output: signature `sig`

1: Parse masked witness `x` from `sk`; `pk` ← `sk->pk_part`; `mseed_eq` ← $\texttt{pk}[0..2 \cdot \texttt{SEED\_SIZE} - 1]$
2: (`com1`, `key`, `x0`, `u0`, `u1`) ← `BLC_Commit_mask_lut` (`salt`,`preCtx`,`mseed`,`x`)
3: (`alpha0`, `alpha1`) ← `ComputePAlpha_mask`(`mseed_eq`, `com1`, `x0`, `x`, `u0`, `u1`)
4: `com2` ← Hash_3(`alpha0`, `alpha1`); `msg_hash` ← Hash_2(`msg`)
5: `hash` ← Hash_4(`pk`, `com1`, `com2`, `msg_hash`); (`i_star`, `nonce`) ← SampleChallenge(`hash`)
6: `opening` ← `BLC_Open_mask`(`key`, `i_star`)
7: `sig` ← (`salt`, `com1`, `com2`, `alpha1`, `opening`, `nonce`)
8: **return** `sig`

secret-shared signing state.

Offline precomputation phase.

- LUT/table generation for accelerated masked Rijndael calls.
- Construction of one-time PRG-related state.
- Precomputation of one-time round-key-related state.
- Allocation and initialization of persistent precompute context.

Online signing phase.

- Consume prebuilt context for message/challenge-dependent signing work.
- Avoid LUT generation, PRG-state reconstruction, and full-table regeneration.
- Keep the online path focused on protocol-dependent computations only.

An additional engineering constraint is state movement overhead: duplicating large prebuilt states can erase intended latency gains. Therefore, the construction favors reference-style or pointer-style reuse of persistent precompute context rather than whole-table copying.

State-ownership model. The split can be summarized as:

$$\texttt{preCtx} = (\texttt{tweak}, \texttt{table}, \texttt{prg}, \texttt{rk}, \texttt{meta}),$$

where online signing reads the public `preCtx` and still computes a masked secret-dependent intermediate state within the online flow.

4.4 Security and Correctness Considerations

The LUT mode does not alter protocol-level outputs, challenge flow, or API-level signing semantics; it changes only how selected masked Rijndael-based subcomputations are realized and scheduled. Correctness is validated against the baseline non-LUT path across all 36 supported variants and masking orders.

At the masking-proof interface, security claims remain attached to the same signing-level masked layers and output boundaries used by the baseline construction. The acceleration path is treated as a backend-level module substitution rather than a protocol redesign.

The offline context `preCtx` used by accelerated signing is treated as public one-time acceleration material for each standby/pre-sign cycle. It is generated (and regenerated) in a public prebuild phase and contains only backend/LUT/PRG/Rijndael-related precomputed state independent of witness-level secrets. Secret-shared signing state (masked

seeds/tree nodes/witness-derived values) is still generated and consumed in the online masked signing flow. Baseline mode remains the reference path; accelerated mode is an optional optimization.

Why the offline split does not change the masking boundary. The accelerated context `preCtx` is public, one-time acceleration material: it contains the backend/LUT/PRG/Rijndael state, independent of witness secrets, and is regenerated per standby/pre-sign cycle. This makes the usual keyed-LUT memory-exposure concern substantially weaker than in long-term secret-key table designs. The offline/online split, therefore, changes scheduling and state placement, not which values must remain secret or where the protocol output boundary lies.

Accordingly, LUT acceleration should be read as a backend-local optimization: it does not change transcript semantics or verification equations, and the baseline compositional theorem remains the primary formal guarantee. Empirically, LUT mode is therefore evaluated under the same leakage-model assumptions as the baseline path, with TVLA serving as supporting measurement evidence rather than a change in the proof boundary.

4.5 Cost Trade-Offs

The accelerated path introduces a three-dimensional trade-off across **offline precomputation time**, **online latency**, and **memory footprint**.

- **Offline time.** Increases because the LUT material, PRG-related state, and one-time round-key-related state are generated before online use.
- **Online time.** Decreases when online signing avoids table generation, PRG-state setup, and copying large prebuilt-state.
- **Memory.** Increases due to persistent precompute context and LUT-backed state retention.

Core driver: context-reuse ratio. The decisive factor is how many online invocations reuse one precomputed context. LUT mode avoids rebuilding Rijndael/LUT state for every leaf-level operation; the setup cost is paid once per reusable configuration class and then amortized over many online calls within a single `BLC.Commit` flow. Concretely:

- SeedCommit contexts are precomputed once per repetition `e` and reused for all leaf commitments in that repetition.
- GGMTree contexts are precomputed once per tree level `j` and reused for all node derivations at that level.
- PRG LUT contexts are precomputed once per repetition/block configuration and reused for all corresponding seed-expansion calls in that repetition.

When total cost can improve. LUT mode is designed to reduce online latency, but the total cost (offline + online) can also be lower when repeated online invocations are sufficiently numerous to amortize the fixed precomputation. This effect is particularly visible in short variants, where leaf-evaluation counts are substantially larger than in fast variants, so identical precomputed contexts are reused more times.

Parameter sensitivity and resource bottleneck. The benefit of LUT mode depends strongly on the masking orders. At moderate masking orders, the reduction in online latency can offset the extra offline computation and memory usage. However, at higher-masking orders, the cost of precomputation and storage grows quickly and may become the main bottleneck. Therefore, memory is a key resource in LUT mode: it is essentially traded for lower latency.

Design implication. Naive LUT integration may still be slower online if table generation, PRG-state setup, or large state copying remains in the online path. Hence, performance gain depends on the completeness of offline extraction, not on the mere presence of LUTs. Overall, the LUT path preserves outer MQOM v2.1 signing structure and should be read as a scheduling/state-placement optimization under explicit offline state ownership, not as a protocol change.

5 Implementation and Evaluation

5.1 Implementation Setup

Our side-channel evaluation was conducted with a ChipWhisperer Pro measurement setup, with power traces captured through the CW1200 acquisition platform. We used a CW308-STM32F415 target board to execute firmware and collect traces. For large-scale benchmarking and precomputation workloads, experiments were orchestrated on a server equipped with an Intel Xeon Gold 5118 @ 2.30 GHz CPU and 768 GB ECC main memory. Although the server also included an NVIDIA Tesla V100 GPU, we did not use GPU-specific optimization; all reported implementation results are CPU-only.

The MQOM v2.1 implementation in this work is based on the official MQOM v2.1 reference source and is extended to our masked signing pipeline. For the baseline AES-based masked components, we use the `masked_combined.c` implementation by Knarfrank (*Higher-Order-Masked-AES-128*)[1], which builds on prior high-order AES masking results [RP10, CPRR15, ZQZ17, CGPZ16]. This baseline was selected because it already supports AES-128 with arbitrary masking order in a share-based software style, which can be adapted to additional Rijndael variants with moderate engineering effort. While bit-sliced AES implementations are often very efficient in software, adapting a high-order bit-sliced design to multiple Rijndael variants appears less straightforward in our setting.

For the LUT-accelerated path, we adapt the implementation structure from the Annapurna PVS team's *Higher-Order-LUT-PRG* repository[2], following its higher-order LUT-oriented software design and its preprocessing/online split, motivated by prior work on LUT masking and PRG-assisted masking pipelines [AVV22, CGZ19].

5.2 Coverage of MQOM v2.1 Variants

Our current evaluation covers all 36 MQOM v2.1 signing variants induced by the category $\in \{1, 3, 5\}$, field family $\in \{\text{gf2}, \text{gf16}, \text{gf256}\}$, profile $\in \{\text{fast}, \text{short}\}$, and repetition parameter $r \in \{3, 5\}$. The codebase keeps a shared module layout across variants and injects variant-specific parameters through the same build and benchmark workflow.

To avoid variant-specific forks in masked logic, all 36 evaluated variants reuse the same masked module interfaces and differ only in parameter packs (evaluation-domain size, repetition count, batching dimensions, and field backend). This makes cross-variant comparisons cleaner: measured differences are driven mainly by MQOM parameter sets rather than by divergent implementations.

[1] https://github.com/knarfrank/Higher-Order-Masked-AES-128
[2] https://github.com/annapurna-pvs/Higher-Order-LUT-PRG

For each variant, we evaluate the same stack of modes: reference baselines, non-LUT masked signing, and LUT-accelerated signing with offline/online decomposition. Section 5.3 reports the cross-variant end-to-end tables used in the main comparison, while Appendix B retains the selected module-level drill-down tables used to interpret the hotspots below.

5.3 Performance Results

5.3.1 Experimental Metrics and Reporting Conventions

All timing results are reported in million CPU cycles. For each variant, we use the reference-table implementation as the single baseline for normalized comparison. We intentionally do not use AES-NI as the primary baseline, since it is already hardware-accelerated and would not provide a fair software-only comparison point for masked implementations. For LUT mode, we split signing into offline, online, and total cycles, and additionally report LUT precompute memory (MB). Ratios in parentheses are normalized to the corresponding reference table runtime of the same row.

All rows follow the same reporting layout, so entries are directly comparable within the same table. The ratio notation "xk" always uses the same-row reference table runtime as the denominator, preventing cross-variant distortion when parameter sizes differ.

5.3.2 Baseline End-to-End Performance Across Variants

Table 1 summarizes `KeyGen` scaling across all 36 evaluated variants and masking orders. Because LUT acceleration affects only the signing-side modules, `KeyGen` has no separate LUT-versus-non-LUT split; the same masked `KeyGen` path is used throughout. For `Mask-2` and `Mask-3`, the overhead remains moderate for most rows. By `Mask-5` and `Mask-6`, the increase is systematic, with `Mask-4` serving as a clear intermediate transition. Still, the growth remains substantially smaller than the signing-side cost shift observed in the LUT-enabled signing results.

Table 2 gives the corresponding end-to-end baseline `Sign` results across the same 36 evaluated variants. As expected, signing overhead is much more sensitive to the masking order than `KeyGen`, because the signing path repeatedly invokes nonlinear masked subroutines.

For signing-side baseline behavior, Appendix B provides the selected module-level decomposition in Table 5. The dominant contributors are `BLC.Commit` and tree/PRG-related components that motivate moving reusable masked preprocessing out of the online path.

5.3.3 Accelerated-Mode Performance Across Variants

Tables 3 and 4 report LUT-accelerated signing across the same 36 evaluated variants, including offline/online split, total cost, and persistent precompute memory.

The main trend is stable across parameter sets: online latency decreases, while offline cost and memory both increase as LUT order grows (LUT-2 $\rightarrow$ LUT-4). Memory growth is approximately linear with table scale, whereas the dominant rapid growth comes from offline table-generation time. At the same time, for some short-profile parameter sets at masking order 2, the LUT-2 total cost can already be lower than baseline masked signing (see Tables 2 and 4). This is the intended scheduling trade-off of LUT mode: shift cost from online signing to precomputation windows and stored state.

5.3.4 Baseline vs. Accelerated Comparison

We compare only the signing cost, since LUT acceleration affects signing-side submodules only; key generation remains unchanged. Combining the selected module-level appendix

Table 1: KeyGen performance comparison across reference and masked implementations. Since LUT acceleration affects only signing, there is no separate LUT versus non-LUT `KeyGen` path. All cycle values are in 10^6 cycles.

Variant	Ref table	Mask-2	Mask-3	Mask-4	Mask-5	Mask-6
cat1gf16fastr3	2.43	3.68 (x1.5)	5.18 (x2.1)	6.72 (x2.8)	8.49 (x3.5)	10.21 (x4.2)
cat1gf16fastr5	2.36	4.05 (x1.7)	5.13 (x2.2)	6.77 (x2.9)	8.27 (x3.5)	10.64 (x4.5)
cat1gf16shortr3	2.3	3.52 (x1.5)	4.87 (x2.1)	6.31 (x2.7)	7.86 (x3.4)	9.44 (x4.1)
cat1gf16shortr5	2.33	3.56 (x1.5)	4.89 (x2.1)	6.29 (x2.7)	7.85 (x3.4)	9.46 (x4.1)
cat1gf256fastr3	1.93	2.53 (x1.3)	3.29 (x1.7)	4.14 (x2.2)	5.33 (x2.8)	6.02 (x3.1)
cat1gf256fastr5	1.95	2.53 (x1.3)	3.43 (x1.8)	4.11 (x2.1)	5.07 (x2.6)	7.66 (x3.9)
cat1gf256shortr3	3.67	3.7 (x1.0)	5.03 (x1.4)	6.61 (x1.8)	8.23 (x2.2)	10 (x2.7)
cat1gf256shortr5	2.6	3.73 (x1.4)	5.24 (x2.0)	6.62 (x2.5)	8.29 (x3.2)	10.1 (x3.9)
cat1gf2fastr3	7.02	7.8 (x1.1)	8.84 (x1.3)	9.49 (x1.4)	10.43 (x1.5)	11.38 (x1.6)
cat1gf2fastr5	7.09	7.76 (x1.1)	8.71 (x1.2)	14 (x2.0)	10.66 (x1.5)	11.41 (x1.6)
cat1gf2shortr3	6.67	6.93 (x1.0)	7.44 (x1.1)	7.96 (x1.2)	10 (x1.5)	8.96 (x1.3)
cat1gf2shortr5	6.73	7.03 (x1.0)	7.42 (x1.1)	8.05 (x1.2)	9.66 (x1.4)	9.05 (x1.3)
cat3gf16fastr3	9.77	13.88 (x1.4)	18.17 (x1.9)	23 (x2.4)	28.38 (x2.9)	33.12 (x3.4)
cat3gf16fastr5	9.66	13.76 (x1.4)	18.56 (x1.9)	22.88 (x2.4)	30.66 (x3.2)	35.06 (x3.6)
cat3gf16shortr3	9.69	13.48 (x1.4)	17.69 (x1.8)	22.17 (x2.3)	29.94 (x3.1)	31.35 (x3.2)
cat3gf16shortr5	9.98	13.49 (x1.4)	17.74 (x1.8)	23.67 (x2.4)	26.9 (x2.7)	31.52 (x3.2)
cat3gf256fastr3	8.66	10.4 (x1.2)	13.08 (x1.5)	14.42 (x1.7)	16.5 (x1.9)	21.67 (x2.5)
cat3gf256fastr5	8.79	10.49 (x1.2)	13.1 (x1.5)	14.55 (x1.7)	16.88 (x1.9)	19.68 (x2.2)
cat3gf256shortr3	10.66	14.39 (x1.4)	18.66 (x1.8)	22.98 (x2.2)	28.78 (x2.7)	32.37 (x3.0)
cat3gf256shortr5	11.38	15.16 (x1.3)	18.56 (x1.6)	22.76 (x2.0)	27.96 (x2.5)	34.22 (x3.0)
cat3gf2fastr3	34.46	37.25 (x1.1)	38.56 (x1.1)	41.61 (x1.2)	44.09 (x1.3)	46.59 (x1.4)
cat3gf2fastr5	33.83	35.75 (x1.1)	38.42 (x1.1)	41.71 (x1.2)	43.85 (x1.3)	46.17 (x1.4)
cat3gf2shortr3	32.64	33.26 (x1.0)	35.17 (x1.1)	36.17 (x1.1)	40.68 (x1.2)	37.61 (x1.2)
cat3gf2shortr5	32.77	32.8 (x1.0)	34.2 (x1.0)	35.41 (x1.1)	36.6 (x1.1)	38.32 (x1.2)
cat5gf16fastr3	20.65	31.03 (x1.5)	42.1 (x2.0)	53.9 (x2.6)	64.84 (x3.1)	79.22 (x3.8)
cat5gf16fastr5	20.81	31.07 (x1.5)	42.14 (x2.0)	67.11 (x3.2)	65.56 (x3.2)	77.91 (x3.7)
cat5gf16shortr3	20.54	30.33 (x1.5)	40.85 (x2.0)	51.91 (x2.5)	62.69 (x3.1)	73.73 (x3.6)
cat5gf16shortr5	20.42	30.18 (x1.5)	41.26 (x2.0)	52.51 (x2.6)	62.99 (x3.1)	76.93 (x3.8)
cat5gf256fastr3	15.85	19.47 (x1.2)	23.76 (x1.5)	28.77 (x1.8)	33.14 (x2.1)	38.33 (x2.4)
cat5gf256fastr5	15.75	19.67 (x1.2)	23.85 (x1.5)	28.34 (x1.8)	33.73 (x2.1)	40.02 (x2.5)
cat5gf256shortr3	20.02	28.85 (x1.4)	37.95 (x1.9)	47.65 (x2.4)	58.35 (x2.9)	69.89 (x3.5)
cat5gf256shortr5	21.1	29.61 (x1.4)	38.27 (x1.8)	47.9 (x2.3)	60.12 (x2.8)	68.68 (x3.3)
cat5gf2fastr3	61.21	68.34 (x1.1)	72.96 (x1.2)	77.96 (x1.3)	81.65 (x1.3)	90.57 (x1.5)
cat5gf2fastr5	60.25	66 (x1.1)	70.34 (x1.2)	76.05 (x1.3)	83.75 (x1.4)	88.72 (x1.5)
cat5gf2shortr3	59	59.24 (x1.0)	63.06 (x1.1)	67.73 (x1.1)	67.45 (x1.1)	71.23 (x1.2)
cat5gf2shortr5	59.41	60.85 (x1.0)	62.23 (x1.0)	68.4 (x1.2)	69.22 (x1.2)	72.9 (x1.2)

tables (Tables 5–7) with the cross-variant end-to-end results in Tables 2–4 gives a direct comparison between baseline masked mode and LUT-enhanced mode. The same hotspot (`BLC.Commit`) shows substantial online reduction under LUT consumption, at the cost of large offline and memory growth at higher LUT order.

At the deployment level, this supports a split operating model: baseline masked mode for memory-constrained or low-precompute settings, or whenever a simple stateless runtime is preferred; and LUT mode for latency-prioritized settings with available offline windows and sufficient memory budget for precompute-context management.

The main text therefore keeps the four cross-variant end-to-end tables, while Appendix B retains the selected module-level breakdowns used for drill-down analysis.

5.4 TVLA Results

5.4.1 TVLA Methodology and Test Configurations

We use a fixed-vs-random TVLA workflow with Welch's t-test and report first-order leakage using the standard threshold $|t| > 4.5$. For each target, trigger strategy, and acquisition settings, consistency is maintained across implementations, so the resulting traces are

Table 2: Sign performance comparison between reference and baseline masked (non-LUT) implementations. All cycle values are in 10^9 cycles.

Variant	Ref table	Mask-2	Mask-3	Mask-4	Mask-5	Mask-6
cat1gf16fastr3	0.06	0.61 (x10.3)	4.66 (x78.8)	4.6 (x77.8)	10.69 (x180.7)	10.97 (x185.5)
cat1gf16fastr5	0.06	0.5 (x8.4)	3.85 (x65.1)	3.69 (x62.3)	9.33 (x157.5)	9.46 (x159.6)
cat1gf16shortr3	0.08	5.53 (x66.6)	29.46 (x354.7)	27.87 (x335.5)	65.06 (x783.3)	69.6 (x837.9)
cat1gf16shortr5	0.07	4.5 (x60.9)	22.85 (x309.3)	24.46 (x331.1)	57.14 (x773.6)	52.69 (x713.4)
cat1gf256fastr3	0.05	1 (x19.5)	5.92 (x116.3)	6.13 (x120.3)	14.45 (x284.0)	14.63 (x287.4)
cat1gf256fastr5	0.05	0.59 (x11.8)	4.65 (x92.2)	4.47 (x88.7)	10.66 (x211.4)	10.81 (x214.5)
cat1gf256shortr3	0.09	7.87 (x89.4)	36.83 (x418.1)	36.42 (x413.4)	83.03 (x942.6)	89.09 (x1011.3)
cat1gf256shortr5	0.09	5.97 (x68.9)	27.26 (x314.7)	27.22 (x314.3)	64.27 (x742.0)	63.51 (x733.2)
cat1gf2fastr3	0.08	0.51 (x6.7)	3.86 (x50.8)	4.1 (x54.0)	9.61 (x126.6)	9.74 (x128.3)
cat1gf2fastr5	0.08	0.51 (x6.8)	3.86 (x51.3)	3.77 (x50.1)	10.44 (x138.7)	9.75 (x129.6)
cat1gf2shortr3	0.1	4.89 (x50.0)	24.54 (x251.2)	25 (x255.9)	53.54 (x547.9)	52.02 (x532.4)
cat1gf2shortr5	0.09	4.6 (x51.9)	22.73 (x256.5)	22.4 (x252.7)	60.32 (x680.4)	58.47 (x659.5)
cat3gf16fastr3	0.15	4.84 (x33.1)	22.51 (x154.0)	22.37 (x153.1)	56.83 (x388.9)	56.74 (x388.3)
cat3gf16fastr5	0.14	3.55 (x25.6)	20.32 (x146.7)	20 (x144.4)	41.89 (x302.4)	42.35 (x305.8)
cat3gf16shortr3	0.18	25.41 (x139.5)	122.41 (x672.0)	125.33 (x688.0)	299.14 (x1642.2)	268.32 (x1473.0)
cat3gf16shortr5	0.16	22.94 (x142.1)	96.65 (x598.6)	109.75 (x679.7)	234.3 (x1451.1)	233.32 (x1445.1)
cat3gf256fastr3	0.11	6.16 (x54.6)	31.29 (x277.0)	32.02 (x283.5)	70.3 (x622.4)	68.27 (x604.5)
cat3gf256fastr5	0.11	4.91 (x45.6)	22.37 (x207.6)	22.47 (x208.6)	50.33 (x467.2)	55.9 (x518.9)
cat3gf256shortr3	0.23	35.17 (x152.5)	168.42 (x730.1)	172.69 (x748.6)	391.3 (x1696.4)	375.24 (x1626.7)
cat3gf256shortr5	0.2	24.94 (x124.7)	122.7 (x613.7)	130.09 (x650.6)	269.25 (x1346.6)	280.14 (x1401.1)
cat3gf2fastr3	0.22	3.73 (x17.2)	20.54 (x94.4)	20.23 (x93.0)	44.44 (x204.2)	46.93 (x215.7)
cat3gf2fastr5	0.22	3.94 (x17.9)	20 (x90.7)	20.21 (x91.7)	43.28 (x196.3)	47.2 (x214.1)
cat3gf2shortr3	0.28	21.21 (x76.3)	104.77 (x376.8)	99.3 (x357.1)	253.55 (x911.8)	244.39 (x878.9)
cat3gf2shortr5	0.29	21.17 (x71.8)	108.19 (x367.2)	103.29 (x350.5)	248.43 (x843.1)	259.18 (x879.5)
cat5gf16fastr3	0.39	6.5 (x16.7)	30.78 (x79.1)	30.3 (x77.8)	71.7 (x184.2)	73.33 (x188.3)
cat5gf16fastr5	0.38	5.89 (x15.7)	26.81 (x71.3)	28.95 (x76.9)	62.32 (x165.6)	66.45 (x176.6)
cat5gf16shortr3	0.4	37.12 (x92.4)	179.2 (x446.0)	162.06 (x403.3)	375.02 (x933.3)	391.31 (x973.8)
cat5gf16shortr5	0.39	29.98 (x76.5)	142.42 (x363.2)	143.66 (x366.4)	314.3 (x801.6)	329.67 (x840.8)
cat5gf256fastr3	0.23	8.93 (x38.3)	42.19 (x180.9)	42.42 (x181.9)	102.44 (x439.2)	104.72 (x449.0)
cat5gf256fastr5	0.23	6.47 (x27.9)	32.96 (x142.3)	31.95 (x137.9)	72.63 (x313.5)	70.36 (x303.7)
cat5gf256shortr3	0.44	50.48 (x113.9)	232.31 (x524.0)	215.6 (x486.3)	558.04 (x1258.7)	551.15 (x1243.2)
cat5gf256shortr5	0.43	35.74 (x83.9)	184.66 (x433.5)	164.47 (x386.1)	393.81 (x924.5)	404.84 (x950.4)
cat5gf2fastr3	0.62	6.41 (x10.4)	26.06 (x42.1)	28.74 (x46.4)	63.55 (x102.6)	69.65 (x112.4)
cat5gf2fastr5	0.61	6.2 (x10.1)	25.56 (x41.7)	28.23 (x46.0)	66.56 (x108.5)	60.64 (x98.8)
cat5gf2shortr3	0.72	32.01 (x44.7)	141.44 (x197.7)	143.49 (x200.6)	345.49 (x482.9)	337.12 (x471.2)
cat5gf2shortr5	0.72	32.32 (x44.9)	146 (x202.9)	155.63 (x216.2)	341.58 (x474.6)	340.4 (x473.0)

directly comparable.

We evaluate four implementation profiles: `ref_table`, `mask2_off`, `mask2_on`, and `mask3_on`. All plots in this section follow this same profile order.

Target scope and coverage limitations are summarized in Section 5.4.4.

5.4.2 TVLA on Small Gadgets

The small-gadget campaign isolates the four instantiations of the bilinear primitive from Section 3.2.1: `secbilinear_basebase`, `secbilinear_baseext`, `secbilinear_extbase`, and `secbilinear_extext`.

Figure 3 gives the representative main-text layout for `secbilinear_basebase`. Appendix C collects the remaining small-gadget layouts in Figures 7–9, always using the profile order `ref_table`, `mask2_off`, `mask2_on`, and `mask3_on`. This fixed ordering makes same-kernel comparisons easier across field combinations.

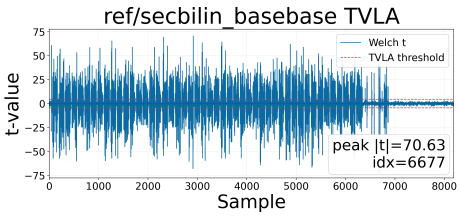
At the interpretation level, the `ref_table` panel is only an unmasked reference for alignment, while the masked panels are the actual first-order targets. These small-gadget figures should be read as an isolation study for the bilinear kernels reused inside `ComputePz_mask` and `ComputePAlpha_mask`: leakage visible here is easy to localize, whereas stable masked panels shift attention in the later composite figures to composition and state reuse.

5.4.3 TVLA on Selected Composite Gadgets

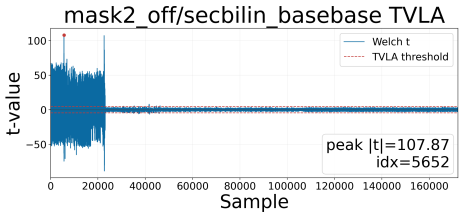
Beyond isolated nonlinear kernels, we also measure three composed targets that exercise the higher-level dataflow used in the signing path: `seedcommit`, `seedderive`, and

Table 3: Signing performance for LUT-based masking variants, highlighting offline and online cost. All cycle values are in 10^9 cycles.

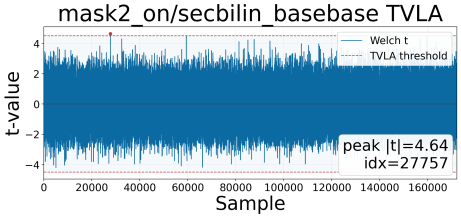
Variant	LUT-2		LUT-3		LUT-4	
	Offline	Online	Offline	Online	Offline	Online
cat1gf16fastr3	2.15 (x36.3)	0.37 (x6.2)	53.95 (x912.3)	0.49 (x8.3)	260.01 (x4396.8)	0.68 (x11.6)
cat1gf16fastr5	2.01 (x34.0)	0.32 (x5.3)	49.93 (x842.6)	0.41 (x7.0)	257.9 (x4352.4)	0.57 (x9.6)
cat1gf16shortr3	1.75 (x21.1)	2.24 (x27.0)	46.88 (x564.5)	3.13 (x37.7)	225.83 (x2719.0)	4.38 (x52.8)
cat1gf16shortr5	1.65 (x22.4)	1.67 (x22.7)	40.85 (x553.0)	2.49 (x33.7)	226.81 (x3070.7)	3.53 (x47.8)
cat1gf256fastr3	2.31 (x45.5)	0.43 (x8.4)	62.04 (x1218.8)	0.66 (x13.0)	301.53 (x5923.7)	0.96 (x18.8)
cat1gf256fastr5	2.02 (x40.0)	0.37 (x7.3)	54.01 (x1071.5)	0.46 (x9.1)	260.49 (x5167.6)	0.66 (x13.0)
cat1gf256shortr3	2.13 (x24.2)	3.06 (x34.8)	48.96 (x555.8)	4.36 (x49.5)	272.96 (x3098.7)	5.4 (x61.3)
cat1gf256shortr5	1.86 (x21.4)	1.98 (x22.9)	44.11 (x509.2)	3.1 (x35.8)	242.61 (x2801.0)	3.9 (x45.0)
cat1gf2fastr3	1.98 (x26.1)	0.35 (x4.6)	46.36 (x610.5)	0.47 (x6.3)	243.26 (x3203.6)	0.6 (x7.9)
cat1gf2fastr5	2 (x26.5)	0.36 (x4.7)	47.3 (x628.4)	0.47 (x6.2)	243.3 (x3232.2)	0.64 (x8.5)
cat1gf2shortr3	1.75 (x17.9)	1.61 (x16.5)	44.1 (x451.3)	2.57 (x26.3)	213.87 (x2188.5)	3.65 (x37.3)
cat1gf2shortr5	1.8 (x20.3)	1.65 (x18.6)	44.26 (x499.3)	2.49 (x28.1)	214.19 (x2416.1)	3.67 (x41.5)
cat3gf16fastr3	9.63 (x65.9)	1.88 (x12.9)	238.78 (x1634.0)	3.11 (x21.3)	1154.47 (x7900.4)	4.09 (x28.0)
cat3gf16fastr5	8.21 (x59.3)	1.44 (x10.4)	207.47 (x1498.1)	2.56 (x18.5)	1150.91 (x8310.4)	3.4 (x24.6)
cat3gf16shortr3	7.81 (x42.9)	11.27 (x61.9)	197.19 (x1082.6)	14.98 (x82.3)	1020.97 (x5605.0)	19.26 (x105.7)
cat3gf16shortr5	6.89 (x42.7)	9.42 (x58.4)	171.59 (x1062.8)	12 (x74.3)	894.2 (x5538.2)	16.12 (x99.8)
cat3gf256fastr3	10.33 (x91.4)	2.61 (x23.1)	259.3 (x2295.7)	3.93 (x34.8)	1433.54 (x12692.2)	5.37 (x47.5)
cat3gf256fastr5	8.92 (x82.8)	1.9 (x17.6)	240.25 (x2230.3)	3.02 (x28.0)	1164.87 (x10814.0)	4.09 (x38.0)
cat3gf256shortr3	8.91 (x38.6)	16.17 (x70.1)	205.24 (x889.7)	20.73 (x89.9)	1147.22 (x4973.4)	26.17 (x113.5)
cat3gf256shortr5	7.82 (x39.1)	10.67 (x53.4)	183.86 (x919.6)	15.21 (x76.1)	1016.78 (x5085.4)	19.28 (x96.4)
cat3gf2fastr3	8.86 (x40.7)	1.57 (x7.2)	205.53 (x944.6)	2.72 (x12.5)	1080.3 (x4965.1)	3.84 (x17.6)
cat3gf2fastr5	8.91 (x40.4)	1.65 (x7.5)	206.44 (x936.3)	2.76 (x12.5)	1148.93 (x5211.1)	3.85 (x17.5)
cat3gf2shortr3	6.85 (x24.7)	9.03 (x32.5)	172.9 (x621.8)	12.34 (x44.4)	894.28 (x3215.9)	16.1 (x57.9)
cat3gf2shortr5	7.39 (x25.1)	9.31 (x31.6)	174 (x590.5)	13.36 (x45.4)	959.37 (x3255.7)	16.1 (x54.6)
cat5gf16fastr3	11.87 (x30.5)	3.08 (x7.9)	325.76 (x836.7)	5.04 (x12.9)	1662.71 (x4270.8)	6.39 (x16.4)
cat5gf16fastr5	11.85 (x31.5)	2.5 (x6.6)	294.41 (x782.4)	4.2 (x11.2)	1437.23 (x3819.7)	5.56 (x14.8)
cat5gf16shortr3	10.92 (x27.2)	16.26 (x40.5)	256.81 (x639.1)	21.51 (x53.5)	1408.33 (x3504.8)	28.95 (x72.0)
cat5gf16shortr5	9.54 (x24.3)	13.57 (x34.6)	239.96 (x612.0)	18 (x45.9)	1332.05 (x3397.3)	23.39 (x59.6)
cat5gf256fastr3	14.68 (x63.0)	4.03 (x17.3)	369.2 (x1583.0)	5.88 (x25.2)	1780.92 (x7636.2)	7.84 (x33.6)
cat5gf256fastr5	11.96 (x51.6)	2.77 (x11.9)	303.23 (x1308.8)	4.25 (x18.3)	1556 (x6716.1)	5.82 (x25.1)
cat5gf256shortr3	12.25 (x27.6)	23.1 (x52.1)	309.09 (x697.2)	29.4 (x66.3)	1479.62 (x3337.5)	37.18 (x83.9)
cat5gf256shortr5	10.22 (x24.0)	15.9 (x37.3)	254.62 (x597.7)	23.15 (x54.3)	1327.38 (x3116.1)	29.42 (x69.1)
cat5gf2fastr3	11.02 (x17.8)	3.03 (x4.9)	276.34 (x446.1)	4.74 (x7.7)	1435.57 (x2317.5)	6.79 (x11.0)
cat5gf2fastr5	11.83 (x19.3)	3.12 (x5.1)	294.43 (x479.9)	5.04 (x8.2)	1530.79 (x2495.2)	6.52 (x10.6)
cat5gf2shortr3	9.54 (x13.3)	14.55 (x20.3)	257.54 (x360.0)	18.33 (x25.6)	1233.58 (x1724.2)	25.45 (x35.6)
cat5gf2shortr5	9.53 (x13.2)	13.87 (x19.3)	258.07 (x358.6)	19.64 (x27.3)	1245.96 (x1731.2)	24.57 (x34.1)



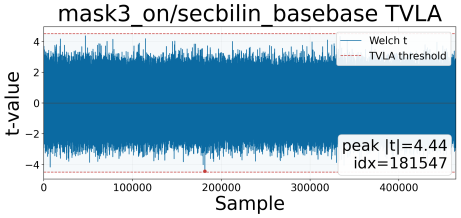
(a) `ref_table`



(b) `mask2_off`



(c) `mask2_on`



(d) `mask3_on`

Figure 3: TVLA results for `secbilinear_basebase`, i.e., the base/base instantiation of the small bilinear gadget, across the four implementation profiles.

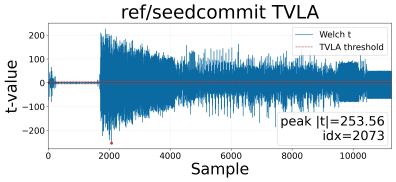
`piop_pz_kernel`. Figures 4–6 present these detailed 2×2 panels in the main text, using the same four-profile layout as Figure 3 and the remaining small-gadget figures, so that the visual comparison remains straightforward as the reader moves from isolated primitives to larger routines.

The first two targets cover the seed-commitment and seed-derivation layers within

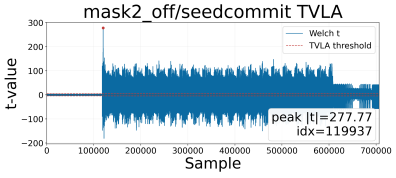
Table 4: Signing performance for LUT-based masking variants, highlighting total cost and memory usage. Cycle values are in 10^9 cycles; memory values are in MB.

Variant	LUT-2		LUT-3		LUT-4	
	Total	Mem	Total	Mem	Total	Mem
cat1gf16fastr3	2.52 (x42.6)	18.27	54.44 (x920.5)	27.41	260.7 (x4408.3)	36.58
cat1gf16fastr5	2.33 (x39.3)	16.86	50.34 (x849.5)	25.3	258.47 (x4361.9)	33.76
cat1gf16shortr3	4 (x48.1)	15.87	50.01 (x602.1)	23.81	230.22 (x2771.8)	31.78
cat1gf16shortr5	3.33 (x45.0)	14.88	43.34 (x586.7)	22.33	230.35 (x3118.5)	29.79
cat1gf256fastr3	2.74 (x53.9)	21.08	62.7 (x1231.8)	31.63	302.49 (x5942.5)	42.2
cat1gf256fastr5	2.38 (x47.3)	18.27	54.47 (x1080.6)	27.41	261.14 (x5180.6)	36.58
cat1gf256shortr3	5.19 (x59.0)	17.86	53.32 (x605.3)	26.79	278.36 (x3159.9)	35.75
cat1gf256shortr5	3.84 (x44.3)	15.87	47.21 (x545.0)	23.81	246.51 (x2846.0)	31.78
cat1gf2fastr3	2.33 (x30.6)	16.86	46.83 (x616.8)	25.3	243.85 (x3211.5)	33.76
cat1gf2fastr5	2.35 (x31.2)	16.86	47.77 (x634.6)	25.3	243.94 (x3240.7)	33.76
cat1gf2shortr3	3.36 (x34.4)	14.88	46.67 (x477.6)	22.33	217.52 (x2225.9)	29.79
cat1gf2shortr5	3.45 (x38.9)	14.88	46.75 (x527.4)	22.33	217.87 (x2457.6)	29.79
cat3gf16fastr3	11.51 (x78.7)	81.19	241.89 (x1655.3)	121.8	1158.57 (x7928.4)	162.44
cat3gf16fastr5	9.66 (x69.7)	74.95	210.03 (x1516.6)	112.43	1154.31 (x8335.0)	149.95
cat3gf16shortr3	19.08 (x104.7)	66.62	212.17 (x1164.8)	99.94	1040.23 (x5710.8)	133.29
cat3gf16shortr5	16.31 (x101.0)	62.46	183.59 (x1137.0)	93.69	910.32 (x5638.1)	124.96
cat3gf256fastr3	12.94 (x114.5)	93.68	263.22 (x2330.5)	140.54	1438.9 (x12739.7)	187.43
cat3gf256fastr5	10.82 (x100.4)	81.19	243.26 (x2258.3)	121.8	1168.97 (x10852.0)	162.44
cat3gf256shortr3	25.08 (x108.7)	74.95	225.97 (x979.6)	112.43	1173.39 (x5086.9)	149.95
cat3gf256shortr5	18.5 (x92.5)	66.62	199.07 (x995.6)	99.94	1036.06 (x5181.9)	133.29
cat3gf2fastr3	10.43 (x47.9)	74.95	208.25 (x957.1)	112.43	1084.14 (x4982.8)	149.95
cat3gf2fastr5	10.55 (x47.9)	74.95	209.2 (x948.8)	112.43	1152.78 (x5228.6)	149.95
cat3gf2shortr3	15.89 (x57.1)	62.46	185.24 (x666.2)	93.69	910.37 (x3273.8)	124.96
cat3gf2shortr5	16.7 (x56.7)	62.46	187.36 (x635.8)	93.69	975.48 (x3310.4)	124.96
cat5gf16fastr3	14.95 (x38.4)	108.26	330.8 (x849.7)	162.4	1669.1 (x4287.2)	216.59
cat5gf16fastr5	14.35 (x38.1)	99.93	298.61 (x793.6)	149.91	1442.79 (x3834.4)	199.93
cat5gf16shortr3	27.18 (x67.6)	92.53	278.32 (x692.6)	138.8	1437.28 (x3576.9)	185.12
cat5gf16shortr5	23.11 (x58.9)	86.75	257.97 (x657.9)	130.13	1355.43 (x3457.0)	173.55
cat5gf256fastr3	18.72 (x80.3)	124.91	375.07 (x1608.2)	187.39	1788.76 (x7669.8)	249.91
cat5gf256fastr5	14.73 (x63.6)	108.26	307.48 (x1327.2)	162.4	1561.82 (x6741.2)	216.59
cat5gf256shortr3	35.34 (x79.7)	104.09	338.49 (x763.5)	156.16	1516.8 (x3421.3)	208.26
cat5gf256shortr5	26.12 (x61.3)	92.53	277.77 (x652.1)	138.8	1356.81 (x3185.1)	185.12
cat5gf2fastr3	14.05 (x22.7)	99.93	281.09 (x453.8)	149.91	1442.36 (x2328.5)	199.93
cat5gf2fastr5	14.94 (x24.4)	99.93	299.47 (x488.1)	149.91	1537.32 (x2505.8)	199.93
cat5gf2shortr3	24.1 (x33.7)	86.75	275.87 (x385.6)	130.13	1259.02 (x1759.8)	173.55
cat5gf2shortr5	23.4 (x32.5)	86.75	277.72 (x385.9)	130.13	1270.54 (x1765.4)	173.55

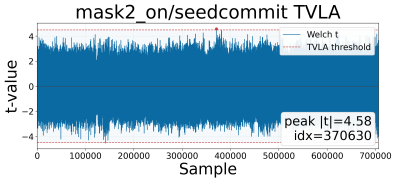
`BLC_Commit_mask`. At the same time, `piop_pz_kernel` captures a representative arithmetic kernel from `ComputePz_mask` at the kernel granularity, since the full reference path does not fit on the measurement device. Together, these main-text figures bridge isolated gadget validation and end-to-end timing by checking whether composition, buffering, and public-output boundaries reintroduce leakage.



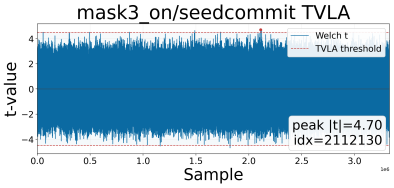
(a) `ref_table`



(b) `mask2_off`

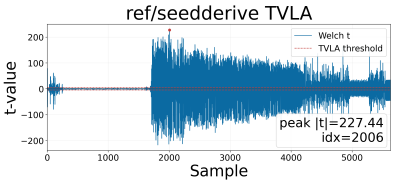


(c) `mask2_on`

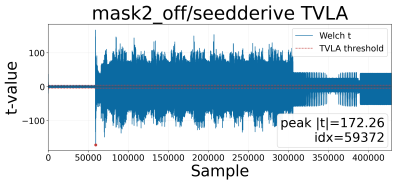


(d) `mask3_on`

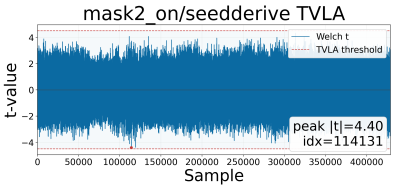
Figure 4: TVLA results for the composed target `seedcommit` across the four implementation profiles. This target captures the masked seed-commitment layer used inside the commitment path.



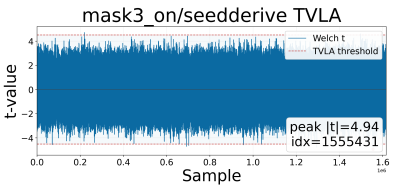
(a) `ref_table`



(b) `mask2_off`

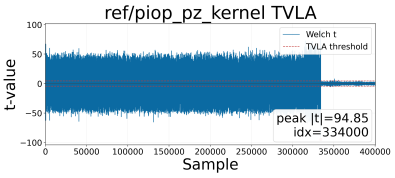


(c) `mask2_on`

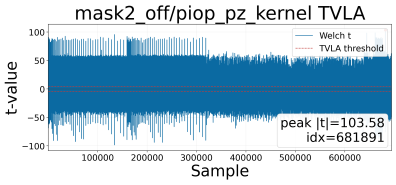


(d) `mask3_on`

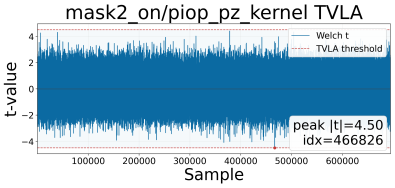
Figure 5: TVLA results for the composed target `seedderive` across the four implementation profiles. This target focuses on the derivation layer reused throughout the masked tree-expansion pipeline.



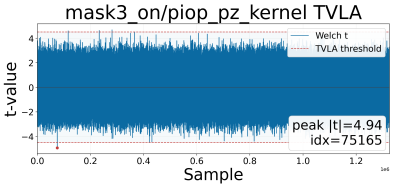
(a) `ref_table`



(b) `mask2_off`



(c) `mask2_on`



(d) `mask3_on`

Figure 6: TVLA results for the representative composed arithmetic target `piop_pz_kernel` across the four implementation profiles. This kernel acts as a proxy for the dominant `ComputePz_mask` arithmetic loop under the device memory constraint.

5.4.4 TVLA Coverage and Limitations

Our TVLA campaign targets critical kernels rather than the entire signing implementation. Measured small gadgets are: `secbilinear_basebase`, `secbilinear_baseext`, `secbilinear_extbase`, and `secbilinear_extext`. Measured composite gadgets are: `seedcommit`, `seedderive`, and `piop_pz_kernel`.

For `piop_pz_kernel`, we test one representative loop-iteration kernel instead of the full reference path: on `STM32F415`, the complete MQOM v2.1 reference code does not fit in available memory. This kernel-level setup is used as a practical proxy for the dominant `ComputePz`-side arithmetic workload under the device constraint.

6 Conclusion

We have presented the first high-order fully-shared masking construction for MQOM v2.1 signing. At the construction level, we provide a baseline masked design that follows the MQOM v2.1 signing pipeline end-to-end and is accompanied by a compositional security argument in the standard probing leakage model. This establishes a concrete reference point for side-channel-resilient MQOM v2.1 implementations.

At the implementation level, we complement the formal analysis with empirical evidence across all 36 MQOM v2.1 signing variants, including timing results and TVLA-based leakage evaluation on representative small-gadget and selected composite-gadget targets. Together, these results provide both proof-level and measurement-level support for the practical viability of the baseline fully-shared design.

To mitigate the dominant online bottleneck, we further introduce an optional Rijndael LUT-based mode that offloads computation to an offline precomputation phase, significantly reducing online signing latency in resource-rich settings. These gains come at the cost of increased offline time and persistent memory, making the choice of mode deployment dependent.

Overall, this work has bridged the gap between provable masking guarantees and deployable MQOM v2.1 signing implementations, enabling practitioners to navigate a clear, explicit security-performance trade-off within a unified framework.

A Formal Security Proofs and Composition

A.1 Primitive Assumptions

This subsection corresponds to the construction-level assumptions in Section 3.1. We adopt the probing model and security notions from Section 2.2. In this section, we only fix the notation used by the formal security proofs: for any probe set P, $\mathsf{View}_P(\mathbf{x};\mathbf{r})$ denotes the corresponding observed joint distribution. Construction-level correctness arguments are intentionally kept in Section 3.

Our small-gadget proofs use the following assumptions.

- Fresh randomness: each call samples independent uniform field elements.
- Probe semantics: probes reveal exact wire values in the considered execution.
- Local composability interface: gadget-level security is stated on explicit input/output shares.
- Randomness boundary: the main arithmetic proof treats fresh masks as an external resource.

The proof strategy is simulator-based: for each admissible probe set, we construct a simulator whose input-share budget is bounded by t.

A.2 Proof-map for Security Notions and Linked Functions

For quick navigation from construction statements above to formal arguments below:

- t-PINI proofs: Section A.3.1.
- t-SNI proofs: Section A.3.2 and composition propositions in Sections A.4.1–A.6.4.
- t-NI target points: obtained via the standard implication chain from the above t-SNI/t-PINI results.
- Construction-view anchors in Section 3: Section 3.3.1, Section 3.3.2, Section 3.4.1, Section 3.4.2, Section 3.4.3, Section 3.4.4, Section 3.5.1, Section 3.5.2, Section 3.5.3, Section 3.5.4, Section 3.6.1, and Section 3.6.2.

A.3 Core Gadgets

This subsection gives formal claims for the construction-level core gadgets in Section 3.2.

A.3.1 General bilinear gadget: formal t-PINI proof

Lemma 1(t-PINI of $\mathsf{SecBilin}_B^d$). Let $d = t + 1$, and let $B : U \times V \to W$ be bilinear over fields of characteristic 2. Assume fresh independent masks $r_{a,b} \xleftarrow{\$} W$ for all $0 \le a < b < d$. Then $\mathsf{SecBilin}_B^d$ satisfies t-PINI.

Proof. Correctness follows Section 3.2.1; we prove security only.

For security, we use the standard ISW/CPRR pairwise-randomization argument for bilinear gadgets: each unordered pair (a, b) receives an independent fresh mask $r_{a,b}$ that randomizes the pairwise cross-term contribution and confines it to the two output domains indexed by a and b. Therefore, any probe set of size at most $t = d - 1$ admits a probe-isolating simulator under the PINI definition stated above. Hence $\mathsf{SecBilin}_B^d$ is t-PINI. □

Corollary 1(Four concrete bilinear instances). The following functions instantiate $\mathsf{SecBilin}_B^d$ with the corresponding bilinear maps:

`SecBilinFieldExtExt`,
`SecBilinFieldExtBase`,
`SecBilinFieldBaseExt`,
`SecBilinFieldBaseBase`.

Hence, they inherit t-PINI.

A.3.2 Refresh gadget: formal t-SNI proof

Lemma 2(t-SNI of $\mathsf{RefreshPair}_{\mathbb{F}}^d$ in characteristic 2). Let $d = t+1$, let $\mathbb{F}$ be a field with $\mathrm{char}(\mathbb{F}) = 2$, and let $x \in \mathbb{F}$ be shared as $x = \sum_{i=0}^{d-1} x_i$. Assume fresh masks $r_{a,b} \xleftarrow{\$} \mathbb{F}$ for all unordered pairs (a, b), mutually independent and independent of the input sharing and of all other gadget calls. Define $\mathsf{RefreshPair}_{\mathbb{F}}^d$ by the pairwise updates $x_a \leftarrow x_a + r_{a,b}$ and $x_b \leftarrow x_b + r_{a,b}$ for every $0 \le a < b < d$. In the implementation, these updates are realized using the XOR assignment (`^=`), which is equivalent to field addition in characteristic 2. Then $\mathsf{RefreshPair}_{\mathbb{F}}^d$ satisfies t-SNI.

Proof. Correctness follows Section 3.2.2: because $\mathrm{char}(\mathbb{F}) = 2$, each pairwise mask $r_{a,b}$ is added twice in the recombined sum and therefore cancels. We prove security only.

Fix any probe set P of size at most $t = d - 1$. Each internal randomness variable $r_{a,b}$ affects only the two shared domains indexed by a and b. Since there are d share domains, but at most $d-1$ probes, at least one share index is not fully observed by P. The simulator uses this partially-observed share domain, together with the fresh masks incident to it, to absorb and reprogram the distribution of probes touching that domain. For probe values supported on share domains that do not using this slack domain, the simulator samples the corresponding fresh masks directly, since they are independent of each other and of all other gadgets. Thus, every probed internal value can be simulated while respecting the t-share input/output budget required by t-SNI. Hence $\mathsf{RefreshPair}_{\mathbb{F}}^d$ is t-SNI. □

Corollary 2(Concrete refresh APIs). The implementation exposes typed refresh gadgets rather than a standalone `RefreshPair` API:

- `SecRefreshFieldExtScalar` for extension-field scalars,
- `SecRefreshFieldBaseScalar` for base-field scalars,
- `SecRefreshFieldExtVec` for extension-field vectors,
- `SecRefreshFieldBaseVec` for base-field vectors.

Each scalar API is an instance of $\mathsf{RefreshPair}_{\mathbb{F}}^d$ over the corresponding characteristic-2 field. Each vector API is obtained by calling the corresponding scalar refresh independently on

each coordinate, with fresh randomness independent across coordinates. In particular, `SecRefreshFieldExtVec` inherits t-SNI on shared $\mathbb{K}$-vectors; the same holds for the other three typed refresh APIs.

A.4 Composed Arithmetic Gadgets

A.4.1 `ComputePz_mask`: composition-based security lemma

Proposition. Assume that:

- `SecBilinFieldExtExt`, `SecBilinFieldExtBase`, and `SecBilinFieldBaseExt` satisfy t-PINI,
- `SecRefreshFieldExtVec`, viewed as a t-SNI refresh on shared $\mathbb{K}$-vectors, satisfies t-SNI,
- All linear field operations used in `ComputePz_mask` are public linear maps and probe-safe,
- Each bilinear or refresh invocation uses fresh randomness that is mutually independent across calls and never reused; the linear steps introduce no new randomness.

Then `ComputePz_mask` satisfies t-SNI.

Proof. `ComputePz_mask` is composed as *public linear* $\rightarrow$ *bilinear* $\rightarrow$ *refresh* $\times 2$ $\rightarrow$ *bilinear* $\times 2$ $\rightarrow$ *public linear*. Here, "refresh $\times 2$" denotes two independent calls to `SecRefreshFieldExtVec`, one on $\mathbf{t}_0$ and one on $\mathbf{x}_0$. The only reused sharings are $\mathbf{t}_0$ and $\mathbf{x}_0$, and both pass through `SecRefreshFieldExtVec` before reuse. This prevents cross-gadget probe correlation on the reused share domains. By the standard PINI/SNI composition theorem, Corollary 1, Corollary 2, probe-safe public linearity, and the independence assumptions above, the full composition is t-SNI. □

A.4.2 `ComputePAlpha_mask`: composition-based security lemma

Proposition. Assume that:

- `ComputePz_mask` satisfies t-SNI,
- two final refresh calls on $\mathbf{v}_0$ and $\mathbf{v}_1$, instantiated by `SecRefreshFieldExtVec`, satisfy t-SNI,
- all compression, addition, recombination, and opening steps are public linear and probe-safe,
- all bilinear and refresh invocations use fresh mutually independent randomness.

Then `ComputePAlpha_mask` satisfies t-SNI at the output-opening boundary.

Proof. For each repetition e, `ComputePAlpha_mask` is the composition

$$\begin{aligned}\texttt{ComputePz\_mask} &\rightarrow \text{public linear compression/addition}\\ &\rightarrow \texttt{SecRefreshFieldExtVec}(\mathbf{v}_0)\\ &\rightarrow \texttt{SecRefreshFieldExtVec}(\mathbf{v}_1)\\ &\rightarrow \text{public recombination/opening.}\end{aligned}$$

By closure of t-SNI under probe-safe public linear transformations, the pre-refresh stage remains t-SNI. Applying two independent t-SNI refresh gadgets preserves t-SNI up to the opening boundary. The final recombination/opening is public linear post-processing step, so it introduces no additional leakage-model assumption and does not violate the t-SNI guarantee. Therefore, the opened outputs $\alpha_0[e], \alpha_1[e]$ satisfy the claimed t-SNI security. □

A.5 Tree and Expansion Layers

This subsection gives formal claims for the construction-level tree and expansion layers in Section 3.4.

A.5.1 `EncMask_ctx` wrappers: composition-based security lemma

Proposition. Assume that:

- masked Rijndael backend `rijndael_mask_encrypt_block_with_ctx` is t-SNI,
- masked key schedule `enc_mask_key_sched` yields a t-SNI masked context,
- interface-level share-domain conversions are linear and probe-safe,
- backend randomness is fresh as required.

Then `enc_encrypt_masked_ctx` is t-SNI. Consequently, `enc_encrypt_masked_x2_ctx` and `enc_encrypt_masked_x4_ctx` are also t-SNI.

Proof. `enc_encrypt_masked_ctx` is one backend masked-encryption call wrapped by linear share-domain conversion and public bookkeeping. Thus, it inherits t-SNI from the backend. The x2/x4 routines are independent compositions of single-block wrapper invocations on disjoint inputs/contexts, so they preserve the same claim. □

A.5.2 `SeedDerive_mask`: composition-based security lemma

Proposition. Assume that:

- `EncMask_ctx` satisfies t-SNI,
- `LinOrtho` is a public linear,
- share-wise XOR is linear and probe-safe.

Then `SeedDerive_mask` satisfies t-SNI.

Proof. `SeedDerive_mask` is the composition of a public linear map, a t-SNI encryption wrapper (`EncMask_ctx`), and a share-wise linear XOR. Because the surrounding steps are linear and probe-safe, the gadget inherits t-SNI directly from `EncMask_ctx`. The x2/x4 batched routines are independent parallel instances and inherit the same property. □

A.5.3 `PRG_mask`: derived-layer security proposition

Proposition. Assume that:

- `EncMask_ctx` and its x2/x4 batched variants satisfy t-SNI,
- `LinOrtho` is a public linear,
- Share-wise XOR is linear and probe-safe,
- Each masked-encryption call uses fresh backend randomness.

Then `PRG_mask` satisfies t-SNI up to its output interface.

Proof. For each index j, one PRG block is built from a masked-encryption call under public tweak separation, followed by public linear post-processing and share-wise linear XOR. Hence, each block inherits t-SNI from the masked-encryption primitive. The complete PRG expansion is a sequential composition of independent block instances, with concatenation/truncation only at the output-format level. Therefore, the same t-SNI interface claim holds for `PRG_mask`. The x2/x4 versions are batching-only and do not change the proof. □

A.5.4 `GGMTree_Expand_mask`: composition-based security lemma

Proposition. Assume that:

- `SeedDerive_mask` satisfies t-SNI.

Then `GGMTree_Expand_mask` satisfies t-SNI.

Proof. `GGMTree_Expand_mask` is built exclusively from `SeedDerive_mask` calls, share-wise XORs, and public tweak/key scheduling. No additional nonlinear gadget appears beyond `SeedDerive_mask`. Since XOR is linear and probe-safe, and each derivation call is t-SNI, the whole level-by-level expansion remains t-SNI by sequential composition. The x2/x4 batched paths are optimization-only and are equivalent to parallel independent derivations, so the same argument applies. □

A.6 Commitment and Opening Layers

This subsection gives formal claims for the construction-level commitment/opening layers in Section 3.5.

A.6.1 `SeedCommit_mask_ctx`: composition-based security lemma

Proposition. Assume that:

- `enc_encrypt_masked_x2_ctx` satisfies t-NI,
- the final refresh gadget satisfies t-SNI,
- `LinOrtho` and share-wise XOR are linear and probe-safe.

Then `SeedCommit_mask_ctx` satisfies t-SNI up to its public-output interface.

Proof. The computation of $(\mathbf{c}_1, \mathbf{c}_2)$ from the shared seed is the composition of masked x2 encryption and linear post-processing and is therefore t-NI under the assumptions. A final t-SNI refresh is then applied to both shared blocks before recombination. Thus, as in the output-boundary argument used for `ComputePAlpha_mask`, final refresh upgrades the public output boundary to t-SNI. Therefore, `SeedCommit_mask_ctx` satisfies the claimed t-SNI interface security; `SeedCommit_mask_x2_ctx` inherits the same claim by independent composition. □

A.6.2 `GGMTree_Open_mask`: opening-interface proposition

Proposition. `GGMTree_Open_mask` is treated as the protocol-specified opening interface for the GGM authentication path. Its algorithmic definition and correctness argument are given in Section 3.5.2. Since its purpose is to reveal public opening data, it is outside the scope of gadget-level t-NI/t-SNI/t-PINI claims.

Security note. `GGMTree_Open_mask` itself is an opening routine and is not claimed to satisfy t-NI or t-SNI. However, because it publicly reveals part of the tree state, the preceding gadget producing the shared node array must satisfy t-SNI. In our setting, this role is played by `GGMTree_Expand_mask`; therefore, the relevant security claim is attached to `GGMTree_Expand_mask` before the opening interface is applied.

A.6.3 `BLC_Commit_mask`: security claim and proof

Proposition (security claim). Assume that:

- `PRG_mask` is t-SNI up to its output interface,
- `GGMTree_Expand_mask` is t-SNI,

- `SeedCommit_mask_ctx` is t-SNI up to its public-output interface,
- `SecRefreshFieldBaseVec` is t-SNI,
- parsing/packing/serialization, Gray-code folding with public weights $w_j = 2^j$, and public hashing are probe-safe linear/public operations.

Then `BLC_Commit_mask` satisfies t-SNI at its mixed-output interface. Its outputs are the public commitment com_1 and the retained internal opening state key, where key = (**node**, lsCom, partial_delta_x). Here com_1 is a designated public protocol output, whereas key is signer-internal state passed only to `BLC_Open_mask`.

Proof. `BLC_Commit_mask` is a composition of previously analyzed secure sublayers: root-seed expansion by `PRG_mask` parameterized for the root domain and output length $\tau \cdot$ `SEED_SIZE`, masked tree expansion, masked seed commitment with authorized public output, second masked PRG expansion on leaf seeds, and only public linear/deterministic post-processing afterward. No new nonlinear gadget is introduced at this level.

Each public leaf commitment released through `SeedCommit_mask_ctx` already passes through that subroutine's internal final refresh on the two ciphertext blocks before recombination, so the public lsCom outputs are exposed only at an SNI-safe boundary.

For coefficient generation, the parsed shared leaf values are not accumulated directly by $\sum_i \omega_i \bar{x}_i$ and $\sum_i \omega_i \bar{u}_i$; instead, the implementation uses Gray-code folding. At folding level j, the right input of each pair contributes to $\mathbf{x}_0[e]$ and $\mathbf{u}_0[e]$ with public weight $w_j = 2^j$, while pairwise sums are carried forward to the next level. The final carried values are exactly $\mathbf{accX}[e]$ and $\mathbf{u}_1[e]$. Since all weights and indices are public, this entire fold/accumulate stage is probe-safe public linear processing on masked data.

The computation of $\Delta x[e] = \mathbf{x} + \mathbf{accX}[e]$ is performed in the shared domain. Before recombination, serialization, and truncation to the retained partial_delta_x$[e]$ bytes, a final t-SNI refresh is applied to the shared representation of $\Delta x[e]$. Hence, the subsequent extraction of partial_delta_x$[e]$, and the hash com_1 built from hashLsCom and these retained bytes, are treated as authorized public output from an SNI-safe boundary.

Therefore, the module inherits t-SNI by composition.

Security interpretation of the key. The retained state is an opening-state container, not a standalone cryptographic key primitive. The t-SNI claim of `BLC_Commit_mask` covers the generation of this opening state—including the retained partial_delta_x bytes—at the commitment-layer output boundary.

A.6.4 `BLC_Open_mask`: opening-interface note

Proposition. `BLC_Open_mask` is treated as the protocol-specified opening interface for challenge indices `i_star`. Its algorithmic definition and correctness argument are given in Section 3.5.4. It reveals only the authentication path, challenged leaf commitment, and retained partial-delta data selected by `i_star`; it is not an interface for arbitrary extraction from the key. Because this routine intentionally outputs authorized public opening data, it is outside gadget-level t-NI/t-SNI/t-PINI claims.

Relation to the composite section. The masking guarantee remains attached to the input-producing gadget `BLC_Commit_mask`. Within `Sign_mask`, safety of invoking `BLC_Open_mask` relies on the binding/consistency guarantees of the retained opening state produced by `BLC_Commit_mask` and on the fact that `i_star` is fixed by the Fiat–Shamir challenge computation on the public transcript, rather than chosen adaptively after observing signer internals. Therefore, `BLC_Open_mask` is treated as a protocol-authorized selective opening interface, not as a generic leakage primitive.

A.7 KeyGen_mask and Sign_mask (Formal Security Theorem)

This subsection gives formal claims for the construction-level modules in Section 3.6.

A.7.1 `KeyGen_mask`

Security goal. The module-level goal is to expose t-SNI up to the public output interface. Because `pk` is public, security must hold against up to t internal probes together with visibility of `pk`; t-NI alone is insufficient.

Proposition (security claim). Let $d = t + 1$ be the number of shares. Assume that:

- `xof_mask` is secure up to its masked output interface,
- `SecBilinFieldBaseExt` satisfies t-PINI,
- `SecRefreshFieldBaseVec` and `SecRefreshFieldExtScalar` satisfy t-SNI,
- parsing, packing, serialization, `ExpandEquations` on a public seed prefix, public base-to-extension matrix multiplication by public coefficients, and vector operations are probe-safe linear/public computations.

Then `KeyGen_mask` satisfies t-SNI up to its public output interface.

Proof. The proof is by composition. Masked XOF outputs witness shares together with the public-key seed shares. The public equation-seed prefix of `pk` is directly recombined from the second half of this masked XOF output. This public seed component—equivalently, the public equation-seed output, often denoted `mseed_eq` in the specification—already belongs to the designated public-key output interface, so opening it there is not an additional leakage channel. Under the `xof_mask` output-interface assumption, no extra refresh is required before calling `ExpandEquations` on that public prefix. The only nonlinear secret-dependent operation in algebraic key generation is `SecBilinFieldBaseExt`, because each share-wise value $\mathbf{t}_i[s] = A_i \cdot \mathbf{x}[s]$ is an extension-field vector produced by a public base-to-extension linear map. The witness shares are reused across equation iterations; therefore, witness sharing is refreshed via `SecRefreshFieldBaseVec` before each subsequent nonlinear reuse. For each equation output, a final `SecRefreshFieldExtScalar` is applied to $\mathbf{y}_i$ before recombination, securing the extension-field public-output boundary. Remaining steps are deterministic public derivations or public linear operations: `ExpandEquations` from the recombined `pk` prefix, share-wise public base-to-extension matrix multiplication by A_i, share-wise linear term addition with public b_i, and packing/serialization. At key finalization, the secret key stores masked witness bytes in `sk->x_part` and copies `pk` into `sk->pk_part`. Final recombination is exactly the designated public-key output boundary. Therefore, composition of t-SNI masked expansion, t-PINI bilinear gadgets, t-SNI refresh gadgets at output and reuse boundaries, and probe-safe linear/public operations yields module-level t-SNI.

A.7.2 `Sign_mask`

Security goal. The module-level notion is t-SNI up to the public output interface. Because signature transcript, challenge, and opening data are public, the full preceding computation must remain secure against up to t probes, while maintaining visibility into the released transcript.

Proposition (security claim). Assume that:

- `BLC_Commit_mask` satisfies t-SNI,
- `ComputePAlpha_mask` satisfies t-SNI,
- all refresh gadgets used inside these layers satisfy t-SNI,
- all bilinear gadgets used inside these layers satisfy t-PINI,
- public hashing/challenge sampling/parsing/packing/serialization are outside the masking boundary or probe-safe,

- `BLC_Open_mask` is treated as an authorized opening routine, not as a secret-preserving gadget.

Then `Sign_mask` satisfies t-SNI up to its public output interface.

Proof. The proof is by composition. The only secret-dependent nonlinear computations occur inside previously analyzed masked layers, namely `BLC_Commit_mask` and `ComputePAlpha_mask`. Public hashing/challenge derivation/serialization operates on public data or data designated to become public at that protocol stage. `BLC_Open_mask` is an authorized opening routine; it is not claimed as a standalone secret-preserving gadget. Its safety in this module relies on preceding t-SNI commitment/arithmetic layers. Therefore, the overall signing computation preserves t-SNI up to the public signature boundary.

Theorem (Joint key-generation and signing). Under the stated assumptions on masked encryption primitives, refresh gadgets, bilinear gadgets, and derived masked layers, `KeyGen_mask` and `Sign_mask` satisfy t-SNI up to their public output interfaces.

B Detailed Performance Tables

This appendix collects the selected module-level performance tables referenced in Section 5.3.

Table 5: Module-level comparison for the selected variant using reference table and non-LUT masking implementations. All cycle values are in 10^6 cycles. Selected variant: mqom2_cat1_gf256_fast_r3.

Module	Ref table	Mask-2	Mask-3	Mask-4	Mask-5	Mask-6
BLC.Commit	12.43	969.6 (x78.0)	6368.43 (x512.4)	6155 (x495.3)	14374.93 (x1156.7)	15751.58 (x1267.5)
BLC.Open	0.00055	0.01 (x22.6)	0.01 (x23.8)	0.02 (x41.9)	0.02 (x43.0)	0.03 (x51.4)
ComputePAlpha	59.42	60.66 (x1.02)	87.14 (x1.5)	124.7 (x2.1)	164.42 (x2.8)	207.31 (x3.5)
ComputePz	3.45	3.67 (x1.06)	5.13 (x1.49)	7.23 (x2.1)	9.67 (x2.8)	12.09 (x3.5)
ExpandEquations	0.91	1.02 (x1.12)	1.03 (x1.12)	1.03 (x1.12)	1.02 (x1.12)	1.09 (x1.19)
GGMTree.Expand	0.05	4.39 (x89.6)	19.92 (x406.3)	20.23 (x412.8)	45.92 (x936.9)	45.5 (x928.3)
GGMTree.Open	0.00015	0.000812 (x5.4)	0.001248 (x8.3)	0.002136 (x14.2)	0.003076 (x20.5)	0.002018 (x13.5)
PRG	0.000624	0.03 (x40.8)	0.09 (x140.2)	0.09 (x140.5)	0.19 (x302.8)	0.19 (x302.4)
SeedCommit	0.000814	0.05 (x61.7)	0.17 (x213.5)	0.18 (x216.8)	0.38 (x467.0)	0.39 (x476.0)
SeedDerive	0.000654	0.03 (x43.9)	0.09 (x140.9)	0.09 (x144.5)	0.2 (x299.1)	0.19 (x293.2)
XOF	0.03	0.04 (x1.4)	0.11 (x3.9)	0.16 (x5.7)	0.25 (x9.0)	0.32 (x11.7)

Table 6: Module-level LUT breakdown for the selected variant, highlighting offline and online cost. All cycle values are in 10^6 cycles. Selected variant: mqom2_cat1_gf256_fast_r3.

Module	Ref	LUT-2		LUT-3		LUT-4	
		Offline	Online	Offline	Online	Offline	Online
BLC.Commit	12.43	2475.14	439.92	62439.33	573	301964.35	807.58
GGMTree.Expand	0.05	67.63	3.09	1712.3	4	8280.51	5.41
PRG	0.000624	9.57	0.02	244.18	0.02	1186.62	0.03
SeedCommit	0.000814	19.23	0.04	488.42	0.04	2368.68	0.05
SeedDerive	0.000654	9.66	0.02	244.13	0.03	1190.98	0.03

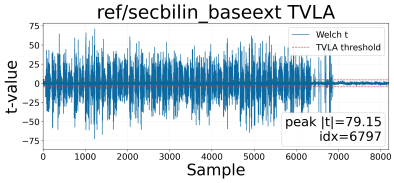
Table 7: Module-level LUT breakdown for the selected variant, highlighting total cost and memory usage. Cycle values are in 10^6 cycles; memory values are in MB. Selected variant: mqom2_cat1_gf256_fast_r3.

Module	Ref	LUT-2		LUT-3		LUT-4	
		Total	Mem	Total	Mem	Total	Mem
BLC.Commit	12.43	2915.06	21.08	63012.33	31.63	302771.93	42.2
GGMTree.Expand	0.05	70.72	0.58	1716.31	0.87	8285.92	1.16
PRG	0.000624	9.59	0.08	244.2	0.12	1186.65	0.17
SeedCommit	0.000814	19.27	0.17	488.46	0.25	2368.74	0.33
SeedDerive	0.000654	9.69	0.08	244.15	0.12	1191.01	0.17

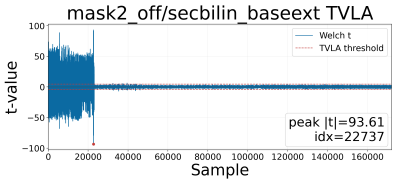
C Detailed TVLA Figures

This appendix collects the remaining TVLA panels referenced in Section 5. Figure 3 in the main text gives the representative base/base small-gadget layout; this appendix keeps the remaining small-gadget panels. All figures use the same profile order: `ref_table`, `mask2_off`, `mask2_on`, and `mask3_on`. When a corresponding plot file is not yet present, the figure keeps a placeholder panel so that the final layout remains stable while images are filled in later.

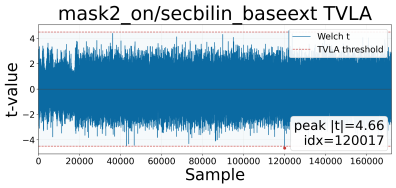
C.1 Small-Gadget TVLA Figures



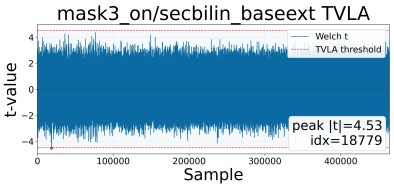
(a) `ref_table`



(b) `mask2_off`

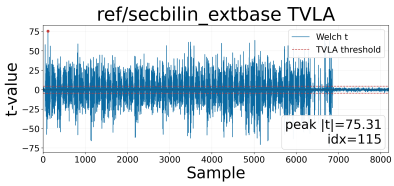


(c) `mask2_on`

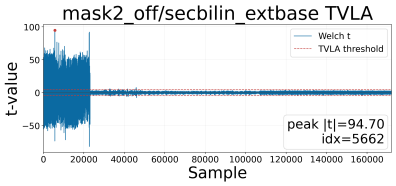


(d) `mask3_on`

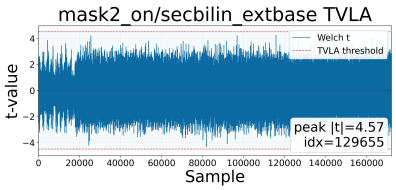
Figure 7: TVLA results for `secbilinear_baseext`, i.e., the base/ext instantiation of the small bilinear gadget, across the four implementation profiles.



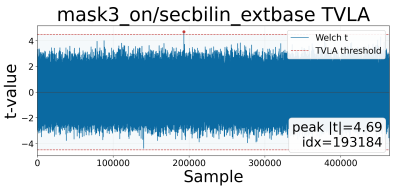
(a) `ref_table`



(b) `mask2_off`

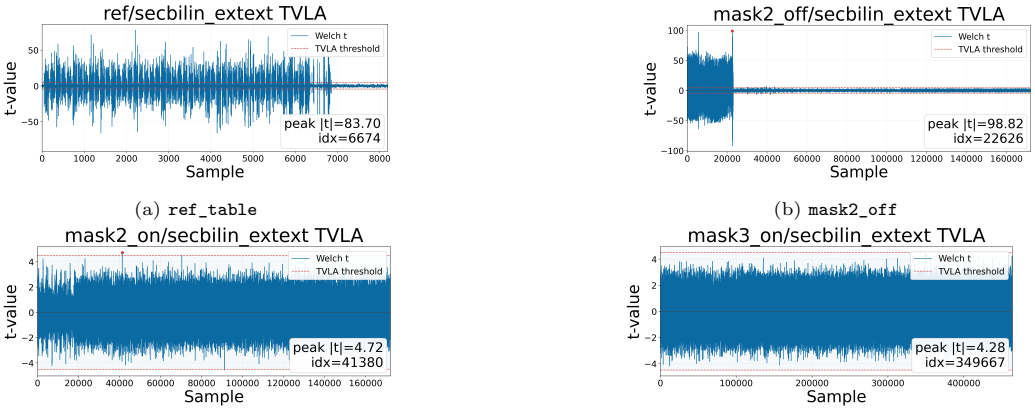


(c) `mask2_on`



(d) `mask3_on`

Figure 8: TVLA results for `secbilinear_extbase`, i.e., the ext/base instantiation of the small bilinear gadget, across the four implementation profiles.



(a) `ref_table` (b) `mask2_off`

(c) `mask2_on` (d) `mask3_on`

Figure 9: TVLA results for `secbilinear_extext`, i.e., the ext/ext instantiation of the small bilinear gadget, across the four implementation profiles.

References

[AAB+25] Gora Adj, Nicolas Aragon, Stefano Barbero, Magali Bardet, Emanuele Bellini, Loïc Bidoux, Jesús-Javier Chi-Domínguez, Victor Dyseryn, Andre Esser, Thibauld Feneuil, Philippe Gaborit, Romaric Neveu, Matthieu Rivain, Luis Rivera-Zamarripa, Carlo Sanna, Jean-Pierre Tillich, Javier Verbel, and Floyd Zweydinger. Mirath signature scheme: Algorithm specifications and supporting documentation. NIST Post-Quantum Cryptography Additional Digital Signature Schemes submission document, February 2025.

[ABB+25a] Najwa Aaraj, Slim Bettaieb, Loïc Bidoux, Alessandro Budroni, Victor Dyseryn, Andre Esser, Thibauld Feneuil, Philippe Gaborit, Mukul Kulkarni, Victor Mateu, Marco Palumbi, Lucas Perin, Matthieu Rivain, Jean-Pierre Tillich, and Keita Xagawa. PERK specification document – round 2. NIST Post-Quantum Cryptography Additional Digital Signature Schemes submission document, February 2025.

[ABB+25b] Nicolas Aragon, Magali Bardet, Loïc Bidoux, Jesús-Javier Chi-Domínguez, Victor Dyseryn, Thibauld Feneuil, Philippe Gaborit, Antoine Joux, Romaric Neveu, Matthieu Rivain, Jean-Pierre Tillich, and Adrien Vinçotte. RYDE signature scheme: Algorithm specifications and supporting documentation. NIST Post-Quantum Cryptography Additional Digital Signature Schemes submission document, February 2025.

[ABC+24] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. Status report on the first round of the additional digital signature schemes for the NIST post-quantum cryptography standardization process. Technical Report NIST IR 8528, National Institute of Standards and Technology, October 2024.

[ABE+21] Diego F. Aranha, Sebastian Berndt, Thomas Eisenbarth, Okan Seker, Akira Takahashi, Luca Wilke, and Greg Zaverucha. Side-channel protections for picnic signatures. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(4):239–282, 2021.

[AVV22] Anju Alexander, Annapurna Valiveti, and Srinivas Vivek. A faster third-order masking of lookup tables. Cryptology ePrint Archive, Paper 2022/1392, 2022.

[BBFR25] Ryad Benadjila, Charles Bouillaguet, Thibauld Feneuil, and Matthieu Rivain. Mqom: Mq on my mind: Algorithm specifications and supporting documentation (version 2.1). MQOM official specification document for the NIST additional digital signature setting, released September 22, 2025, September 2025.

[CDG+17] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1825–1842. ACM, 2017.

[CGPZ16] Jean-Sébastien Coron, Aurélien Greuet, Emmanuel Prouff, and Rina Zeitoun. Faster evaluation of SBoxes via common shares. Cryptology ePrint Archive, Paper 2016/572, 2016.

[CGZ19] Jean-Sébastien Coron, Aurélien Greuet, and Rina Zeitoun. Side-channel masking with pseudo-random generator. Cryptology ePrint Archive, Paper 2019/1106, 2019.

[CPRR15] Jean-Sébastien Coron, Emmanuel Prouff, Matthieu Rivain, and Thomas Roche. Higher-order side channel security and mask refreshing. In *Fast Software Encryption – FSE 2013*, pages 410–424. Springer, 2015.

[FRWJ25] Thibauld Feneuil, Matthieu Rivain, and Auguste Warmé-Janville. Masking-friendly post-quantum signatures in the threshold-computation-in-the-head framework. Cryptology ePrint Archive, Paper 2025/520, 2025.

[GSE20] Tim Gellersen, Okan Seker, and Thomas Eisenbarth. Differential power analysis of the picnic signature scheme. Cryptology ePrint Archive, Paper 2020/267, 2020.

[MBB+25] Carlos Aguilar Melchor, Slim Bettaieb, Loïc Bidoux, Thibauld Feneuil, Philippe Gaborit, Nicolas Gama, Shay Gueron, James Howe, Andreas Hülsing, David Joseph, Antoine Joux, Mukul Kulkarni, Edoardo Persichetti, Tovohery H. Randrianarisoa, Matthieu Rivain, and Dongze Yue. The syndrome decoding in the head (SD-in-the-Head) signature scheme: Algorithm specifications and supporting documentation – version 2.0. NIST Post-Quantum Cryptography Additional Digital Signature Schemes submission document, February 2025.

[Mil86] Victor S. Miller. Use of elliptic curves in cryptography. In *Advances in Cryptology – CRYPTO '85 Proceedings*, pages 417–426. Springer, 1986.

[Nat22] National Institute of Standards and Technology. Status report on the third round of the NIST post-quantum cryptography standardization process. Technical Report NIST IR 8413, National Institute of Standards and Technology, July 2022.

[Nat24a] National Institute of Standards and Technology. FIPS 203: Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standard (FIPS) 203, August 2024.

[Nat24b] National Institute of Standards and Technology. FIPS 204: Module-lattice-based digital signature standard. Federal Information Processing Standard (FIPS) 204, August 2024.

[Nat24c] National Institute of Standards and Technology. FIPS 205: Stateless hash-based digital signature standard. Federal Information Processing Standard (FIPS) 205, August 2024.

[Nat24d] National Institute of Standards and Technology. Nist releases first 3 finalized post-quantum encryption standards. NIST news item, August 2024.

[Nat25a] National Institute of Standards and Technology. HQC announced as a 4th round selection. NIST CSRC news item, March 2025.

[Nat25b] National Institute of Standards and Technology. Post-quantum cryptography: Additional digital signature schemes – round 2 additional signatures. NIST CSRC project page, 2025.

[RP10] Matthieu Rivain and Emmanuel Prouff. Provably secure higher-order masking of AES. In *Cryptographic Hardware and Embedded Systems – CHES 2010*, pages 413–427. Springer, 2010.

[RSA78] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[Sho97] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[ZQZ17] Rui Zhang, Shuang Qiu, and Yongbin Zhou. Further improving efficiency of higher-order masking schemes by decreasing randomness complexity. *IEEE Transactions on Information Forensics and Security*, 12(11):2590–2598, 2017.